

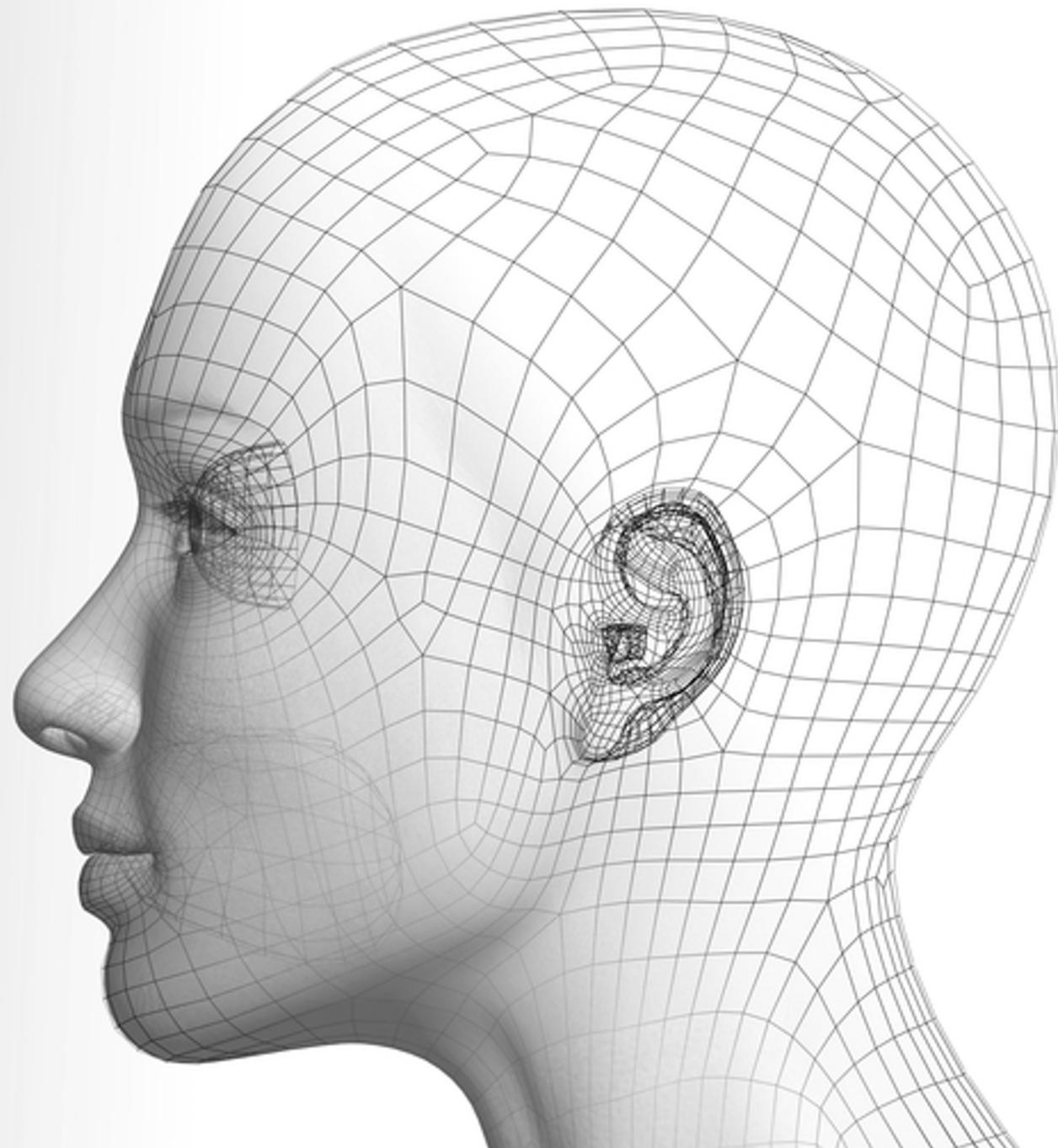
Unsupervised Representation Learning

Chen-Change Loy

吕健勤

<https://www.mmlab-ntu.com/>

<https://twitter.com/ccloy>



Announcement

- Next Friday (15 Apr) is holiday. The last lecture will be converted to e-learning. Not included in the quiz.

Timetable

Week	Date	Topic	Lecturer
1	14 Jan	Introduction	Loy Chen Change
2	21 Jan	Convolutional Neural Networks	Loy Chen Change
3	28 Jan	Training Convolutional Neural Networks	Loy Chen Change
4	4 Feb	Object Detection	Loy Chen Change
5	11 Feb	Image Segmentation	Liu Ziwei
6	18 Feb	Transformers	Loy Chen Change
7	25 Feb	Autoencoder	Liu Ziwei
Recess Week			
8	11 Mar	Generative Adversarial Networks	Liu Ziwei
9	18 Mar	Image Super-Resolution	Loy Chen Change
10	25 Mar	Image Editing	Dr Yang Shuai
11	1 Apr	Open-World Visual Recognition	Dr Zhou Kaiyang
12	8 Apr	Unsupervised Representation Learning	Loy Chen Change
13	E-learning	Creating Small and Efficient Networks	Loy Chen Change

Summary

- Week 13 (11 Apr) – Submission of Homework 2
- Week 13 (14 Apr) – Submission of Presentation for Paper Reading
- Week 14 (22 Apr) – Submission of Project 2
- Week 15 (29 Apr) – Quiz (online)

Quiz

- Date: 29 April (Friday)
- Time: 7:30 PM – 8:30 PM
- Venue: Online at NTULearn
- This quiz will be allocated 60 minutes and it is an open-book quiz, during which you will be required to answer ~~3 text entry questions and 20 multiple choice questions~~ 25 multiple-choice questions. There is no restriction on the use of writing material, printed notes or the Internet. But you are not allowed to communicate with other people via any means during the quiz.

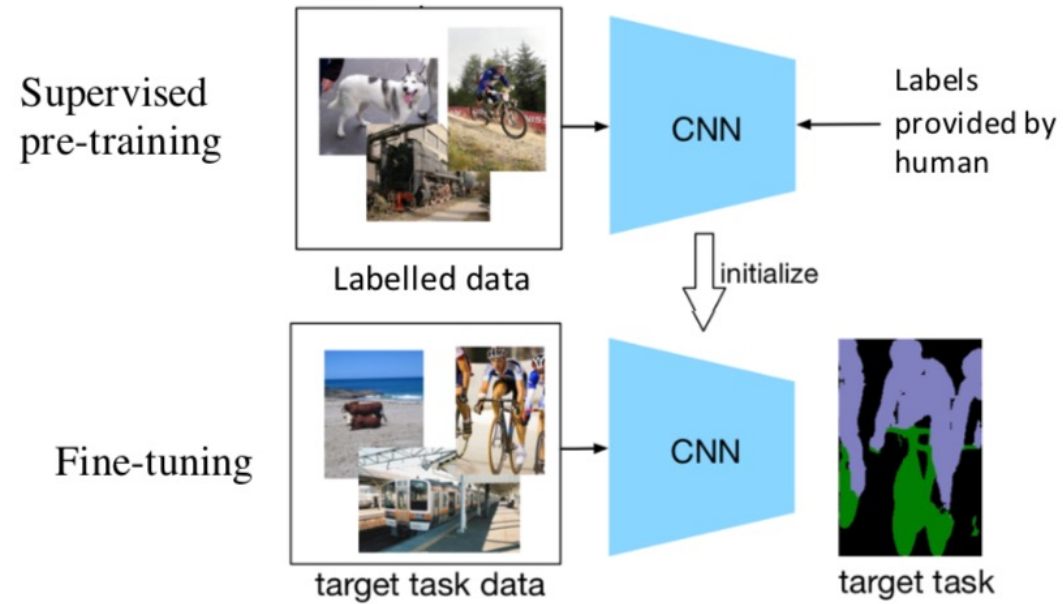
The questions will be based on all lectures.

All questions will add up and normalize to 20 marks.

Outline

- Introduction to self-supervised learning
- Contextual-based methods
- Video-based methods
- Methods based on contrastive learning
- Clustering-based methods

Supervised Pre-training



What is self-supervised (feature) learning?

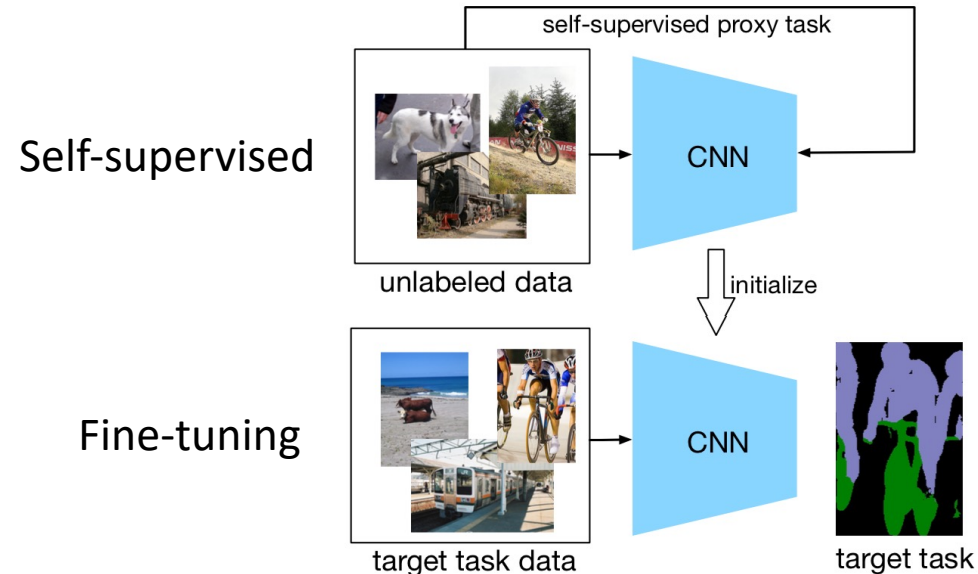
Learning deep representations without extensive annotations

The proxy stage:

- Does not need any labeled data but requires one to design a **proxy or pretext task** with self-derived supervisory signals on unlabeled data
- (Almost) without human intervention

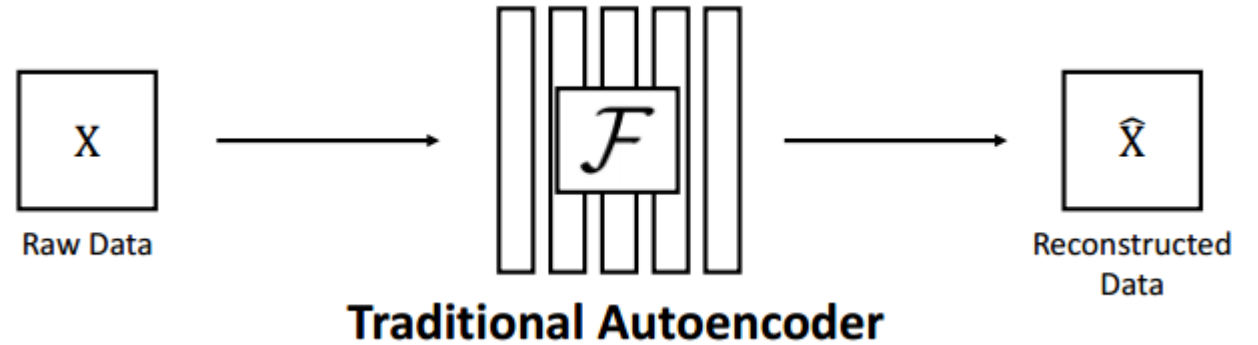
The fine-tuning stage:

- Use the trained network as a starting point for other visual tasks
- The network is fine-tuned using a handful of annotated data from a target domain.



Why self-supervised (feature) learning

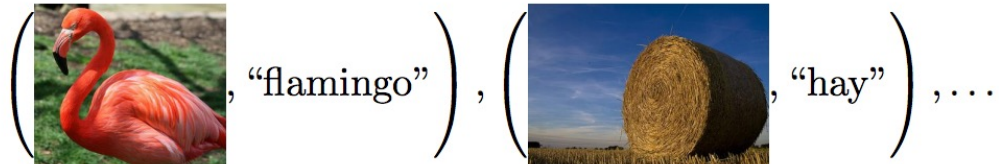
- Problems with autoencoder
 - Use pixel-wise loss – no structural loss incorporated
 - Reconstruction can hardly represent semantic information
- Self-supervised learning (when appropriate pretext task is chosen)
 - Help semantic representation learning with self-labels
 - Can be transferred to other tasks!
- Mask autoencoder, with the right architecture and training strategies, works very well!



Examples of proxy tasks

Learning a representation via (x, y) pairs

Classification

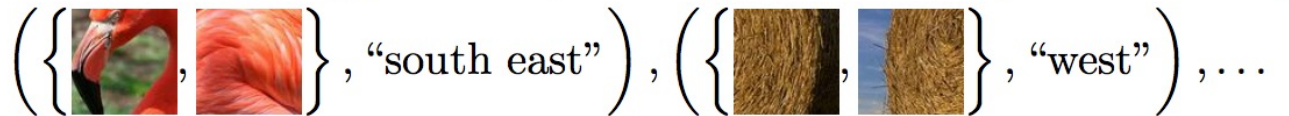


Self-supervision

Ex. 1: **Inpainting** (remove patch and then predict it)



Ex. 2: **Context** (given two patches, predict their spatial relation)



Ex. 3: **Colorization** (predict color given intensity)

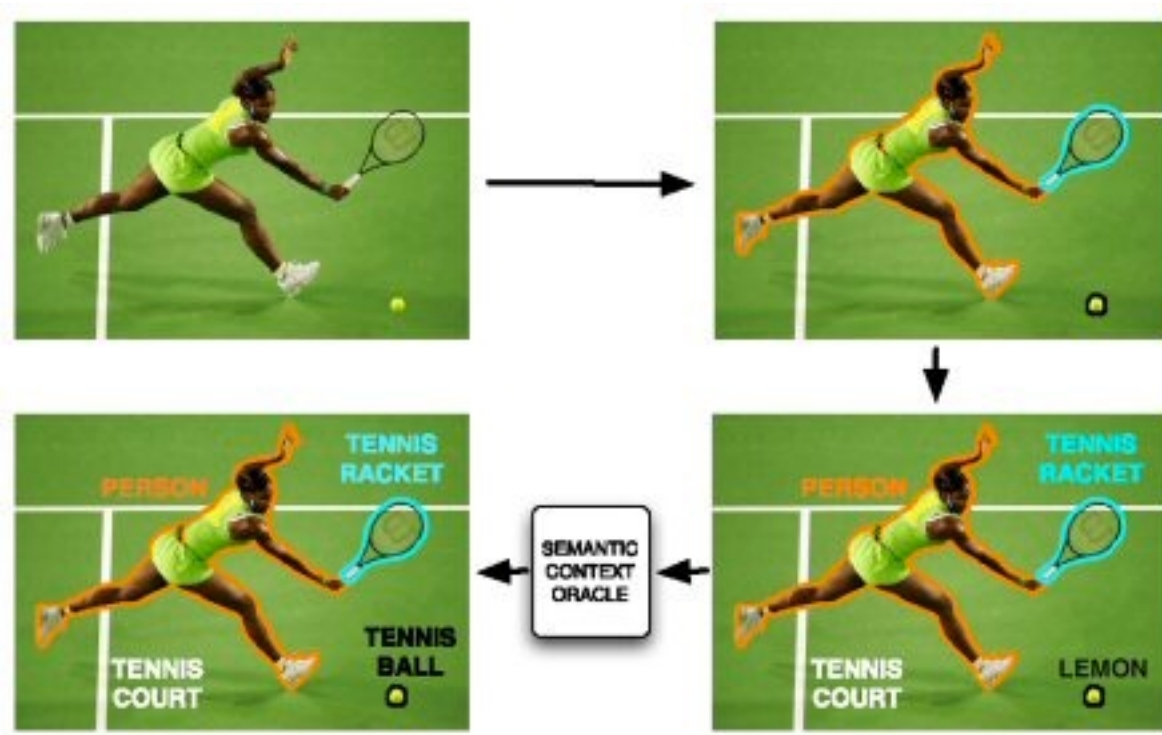


Video, Exemplar, Cross Modality ...

Contextual-based Methods

Context

Context is ubiquitous in CV



Rabinovich et al, Objects in Context, ICCV 2007

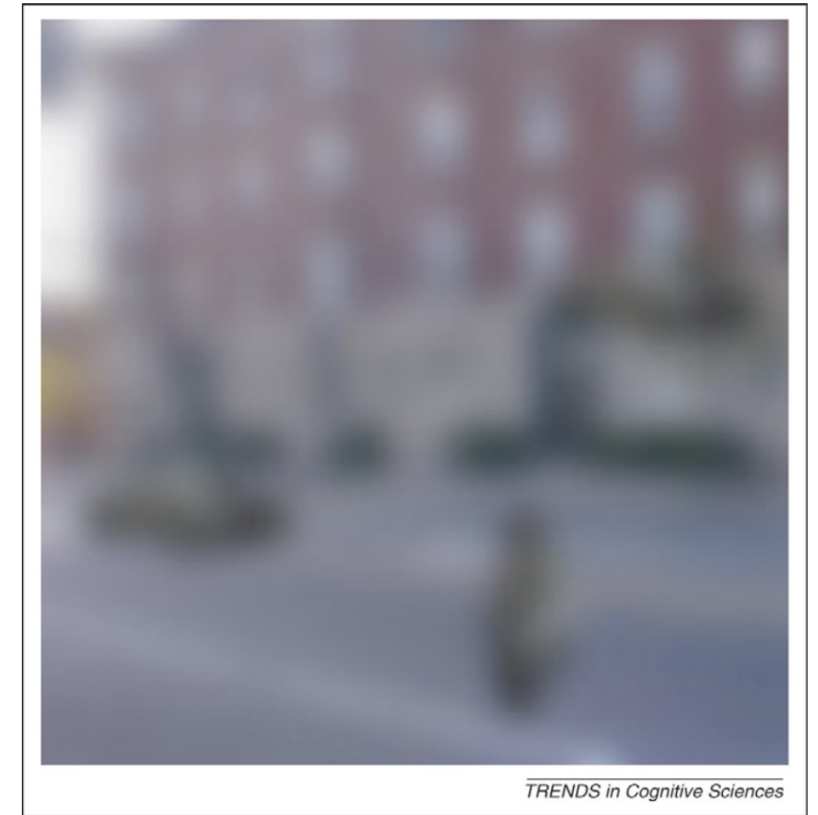


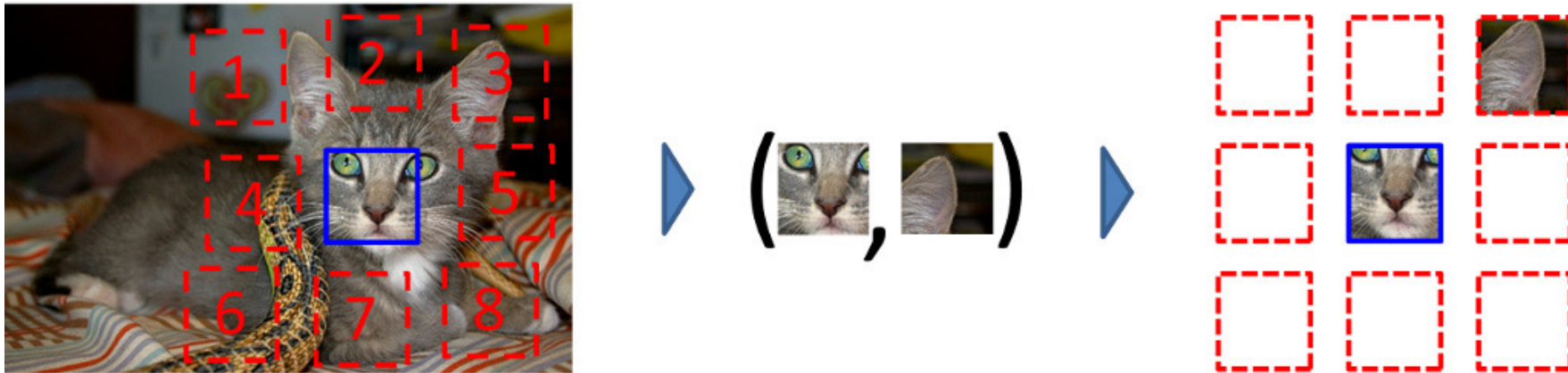
Figure 2. The strength of context. The visual system makes assumptions regarding object identities according to their size and location in the scene. In this picture, observers describe the scene as containing a car and pedestrian in the street. However, the pedestrian is in fact the same shape as the car, except for a 90° rotation. The atypicality of this orientation for a car within the context defined by the street scene causes the car to be recognized as a pedestrian.

Aude Oliva and Antonio Torralba, The role of context in object recognition, 2007

Self-supervised learning by solving jigsaw puzzles

Visual representation is a configuration of parts (geometry) + parts appearance (texture)

Can be learned by solving jigsaw puzzles as a pretext task



Self-supervised learning by solving jigsaw puzzles

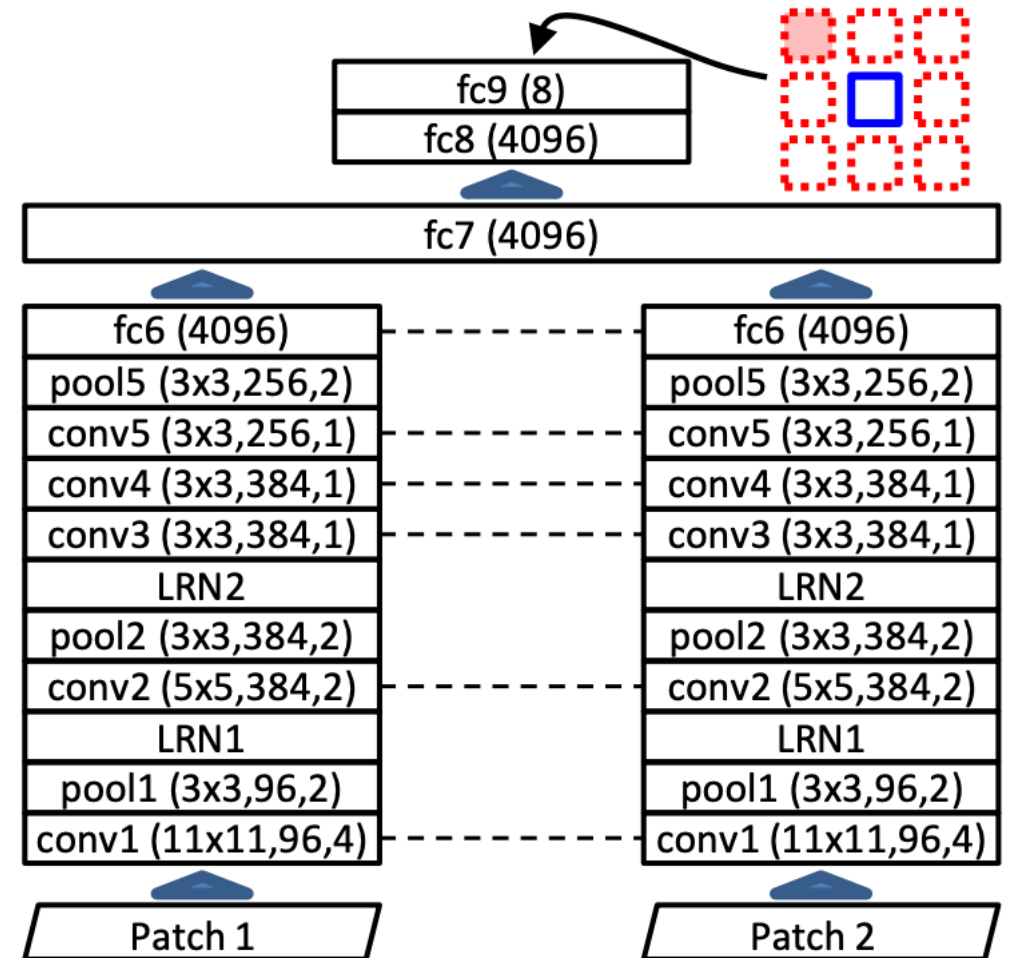
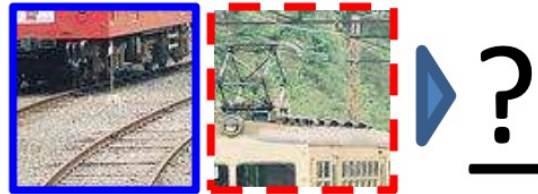
This task begins by sampling two patches at random from a single image and feeding them both to the network without context.

The network's goal is to predict where one patch was relative to the other in the original image.

Question 1:



Question 2:



AlexNet-style architectures

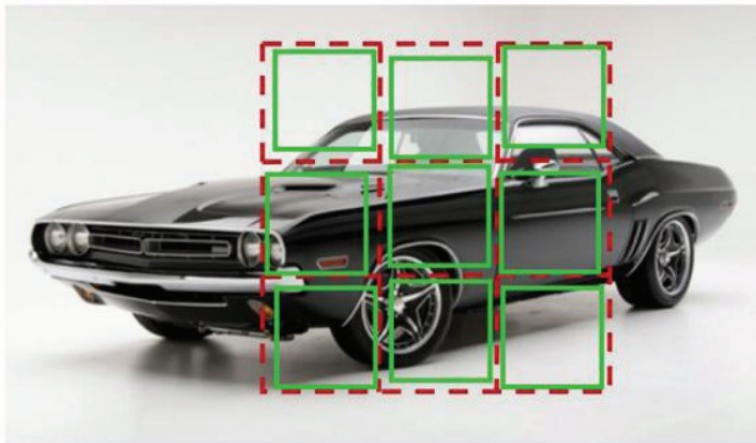
Self-supervised learning by solving jigsaw puzzles



What does one need to learn to solve a jigsaw puzzle?

Self-supervised learning by solving jigsaw puzzles

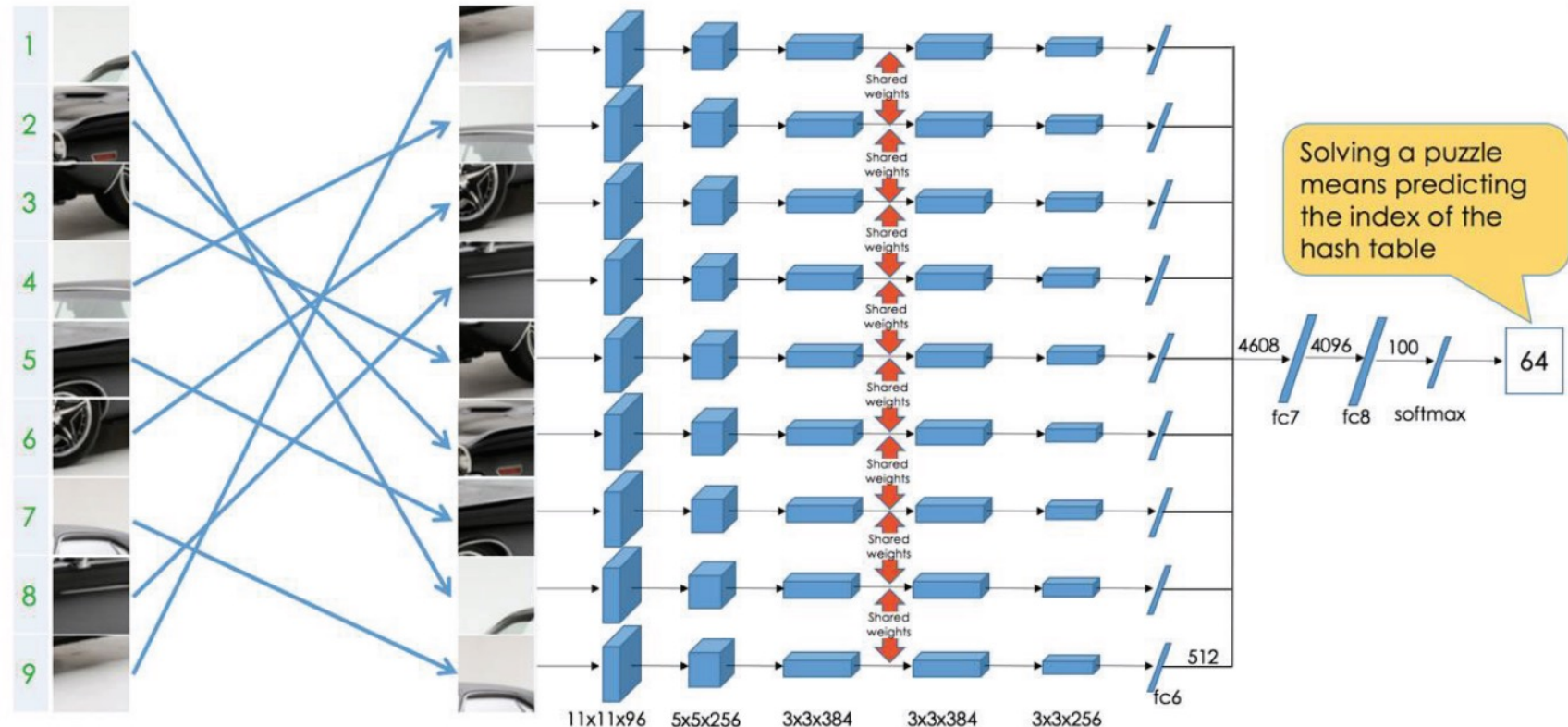
Select a random 3x3 grid and a random 64x64 patch from each 75x75 cell



Hash Set

index	table
64	9,4,6,8,3,2,5,1,7

Reorder patches according to the selected hash table

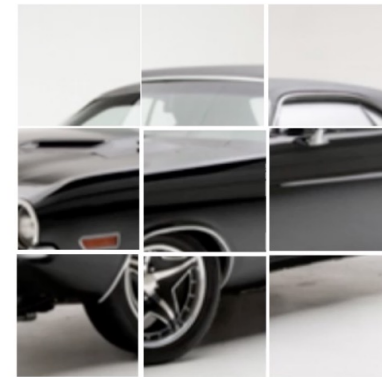
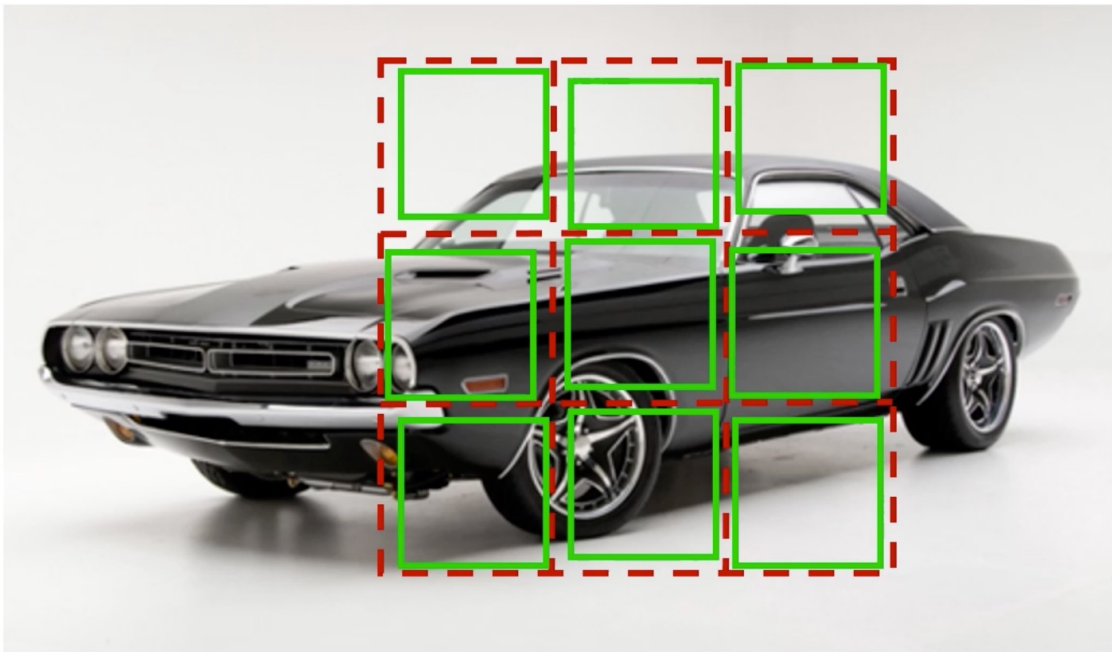


Solving Jigsaw Puzzles using the Context-Free Network (CFN)

Self-supervised learning by solving jigsaw puzzles

Random gaps between patches to avoid “shortcuts” due to edge continuity and pixel intensity distribution

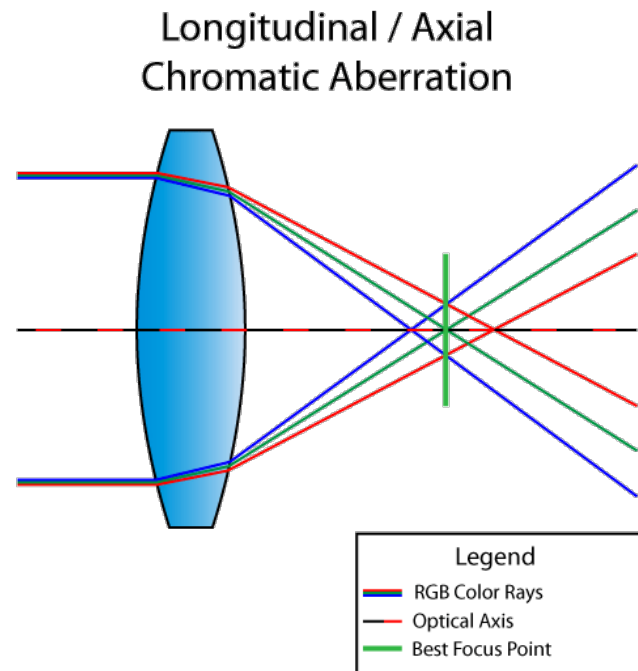
- Prevent the CFN to solve puzzles by matching borders
- Encourage the CFN to learn spatial relationships between patches (object parts)



Self-supervised learning by solving jigsaw puzzles

Jitter color channels and use grayscale images to avoid shortcuts due to chromatic aberration

Chromatic aberration occurs when different wavelengths of color do not converge at the same point after passing through a lens



Chromatic Aberration. Chromatic aberration is a relative spatial shift between color channels that increases from the images center to the borders. This type of distortion helps the network to estimate the tile positions. To avoid this shortcut, we use three techniques: i) We crop the central square of the original image and resize it to 255×255 ; ii) We train the network with both color and grayscale images. Our training set is a composition of grayscale and color images with a ratio of 30% to 70%; iii) We (spatially) jitter the color channels of the color images of each tile randomly by $\pm 0, \pm 1, \pm 2$ pixels.

By Inpainting



(a) Input context



(b) Human artist

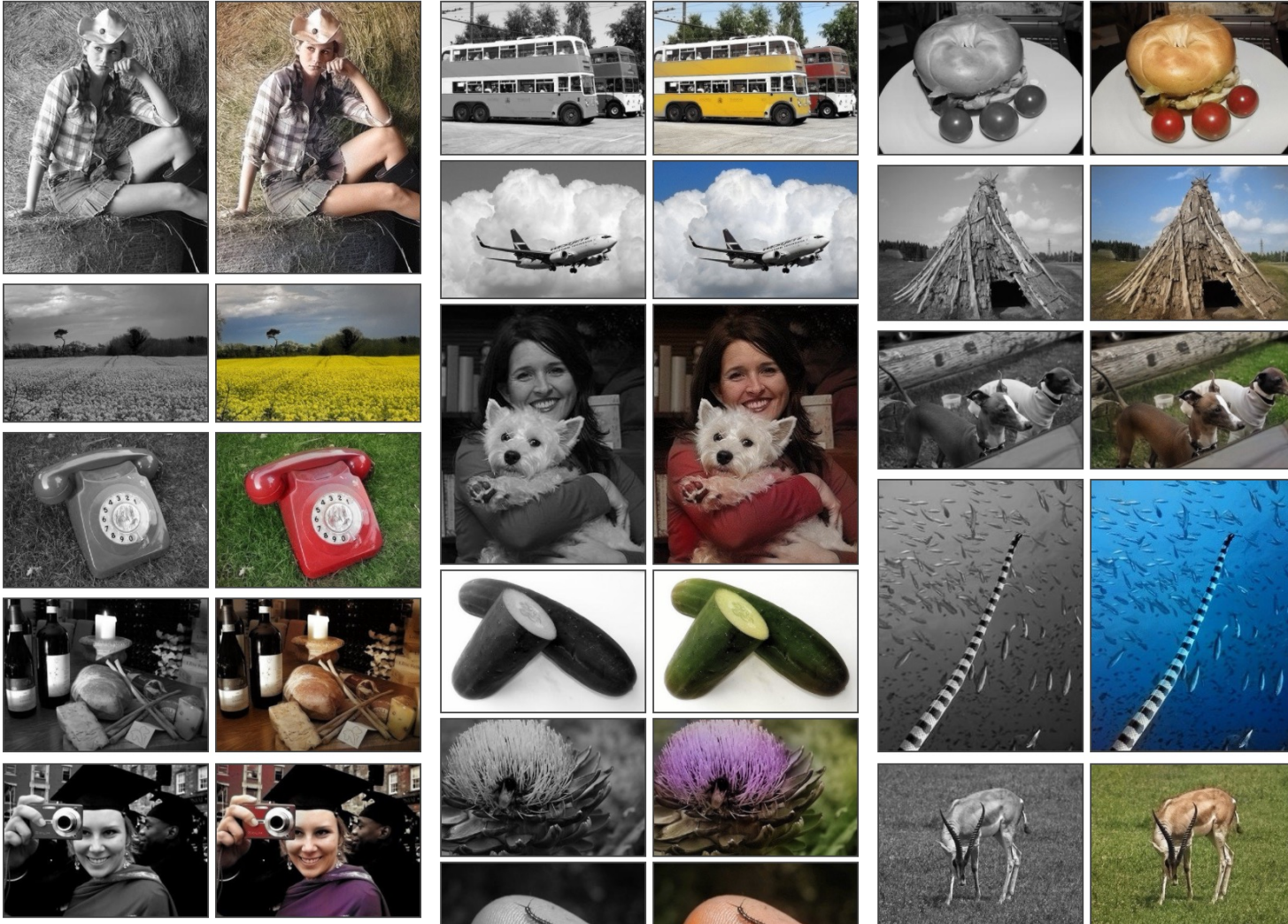


(c) Context Encoder
(L_2 loss)



(d) Context Encoder
(L_2 + Adversarial loss)

Self-supervised learning by colorization



Papers

Larsson, G.; Maire, M.; and Shakhnarovich, G. Learning representations for automatic colorization. ECCV 2016.

Richard Zhang, Phillip Isola, and Alexei A. Efros. Colorful Image Colorization. ECCV 2016

Larsson, G.; Maire, M.; and Shakhnarovich, G. 2017. Colorization as a proxy task for visual understanding, CVPR 2017

Self-supervised learning by colorization

A natural image is composed of luminance channel and chrominance channels.

The proxy task is formulated to predict an image chrominance from the luminance of the same image.

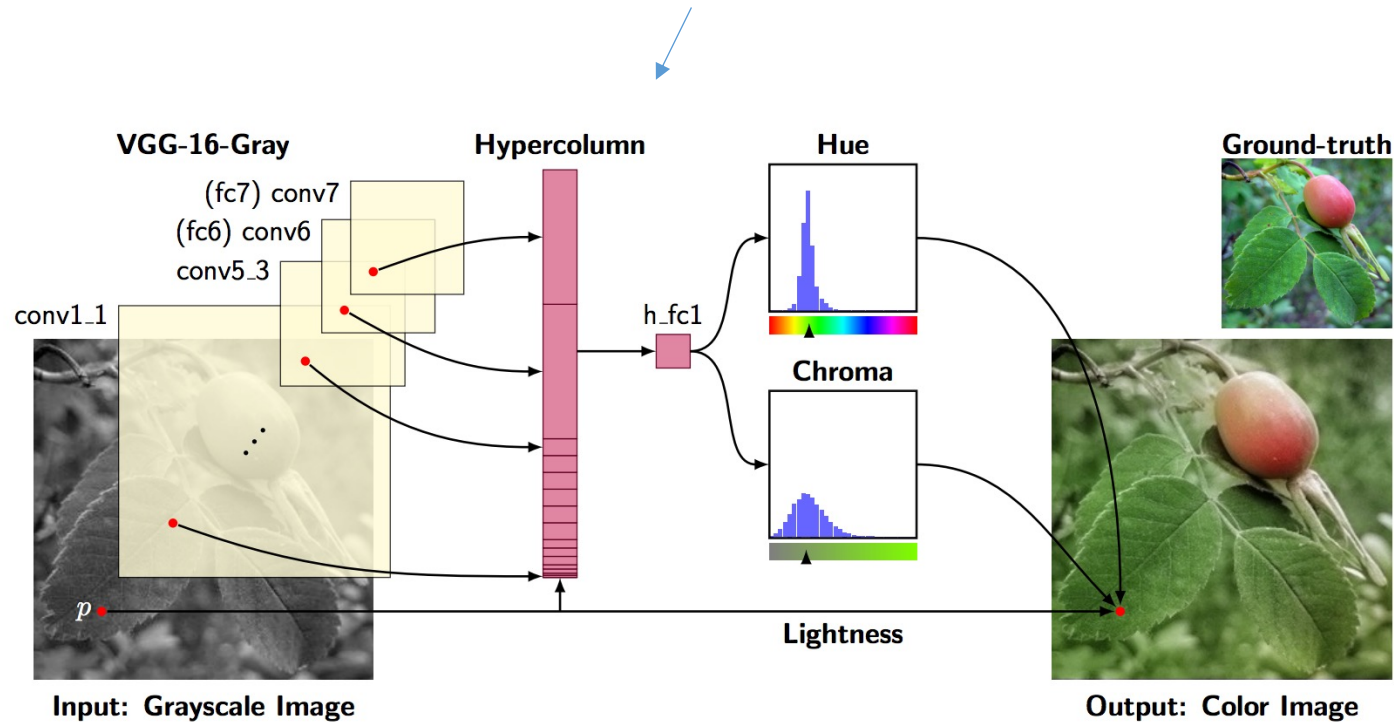
“Colorization is a great proxy task since this task requires high-level visual understanding”



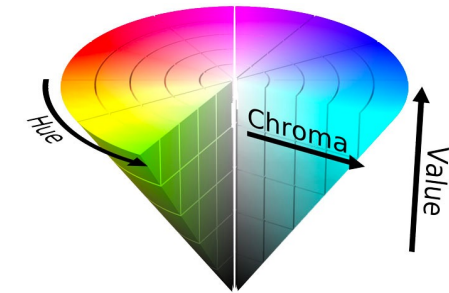
Self-supervised learning by colorization

- [Larsson ECCV 2016] chose hue/chroma with histogram predictions at sparse locations.

spatially localized multilayer slices as per-pixel descriptors

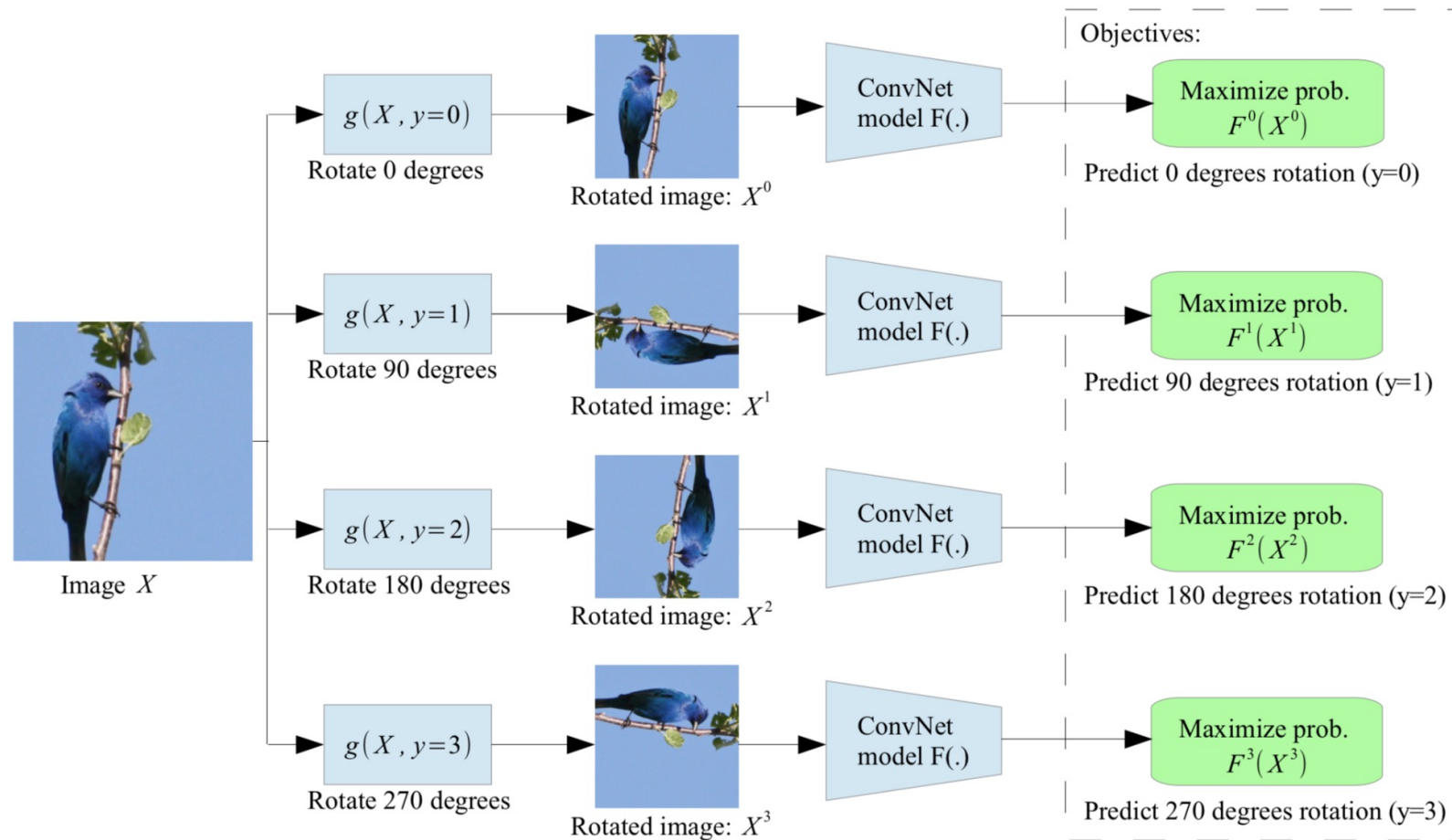


Source: Larsson et al. (2016)



$$L_{\text{hue/chroma}}(\mathbf{x}, \mathbf{y}) = D_{\text{KL}}(\mathbf{y}_C \| f_C(\mathbf{x})) + \lambda_H y_C D_{\text{KL}}(\mathbf{y}_H \| f_H(\mathbf{x}))$$

Rotation



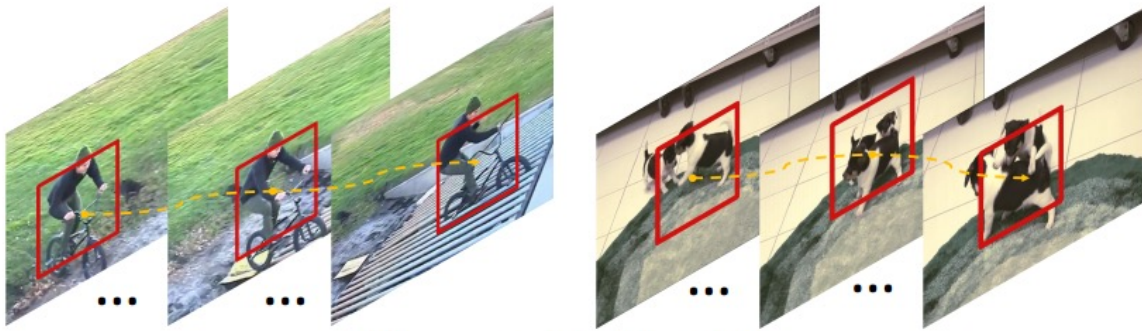
- The model is trained to predict which rotation has been applied, thus a 4-class classification problem
- Drives the model to learn semantic concepts of objects
- The model has to learn to recognize high level object parts, such as heads, noses, and eyes, and the relative positions of these parts

Video-based Methods

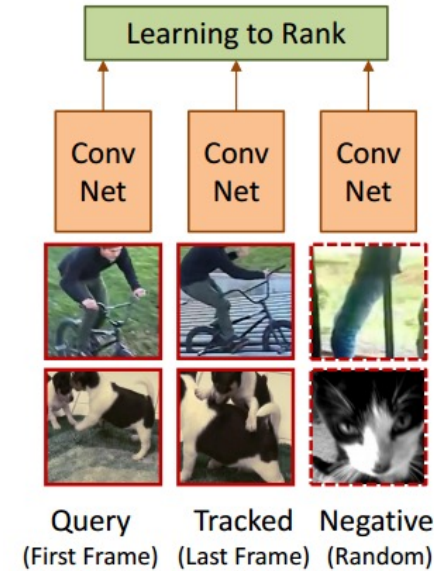
Exploiting video for self-supervised learning

- Can be obtained freely with rich information
 - Temporal continuity
 - Motion consistency
 - Action order
- High-level semantic visual concepts associated with video frames typically change only gradually as a function of the pixels that compose the frames.

Exploiting corresponding pairs



(a) Unsupervised Tracking in Videos



(b) Siamese-triplet Network

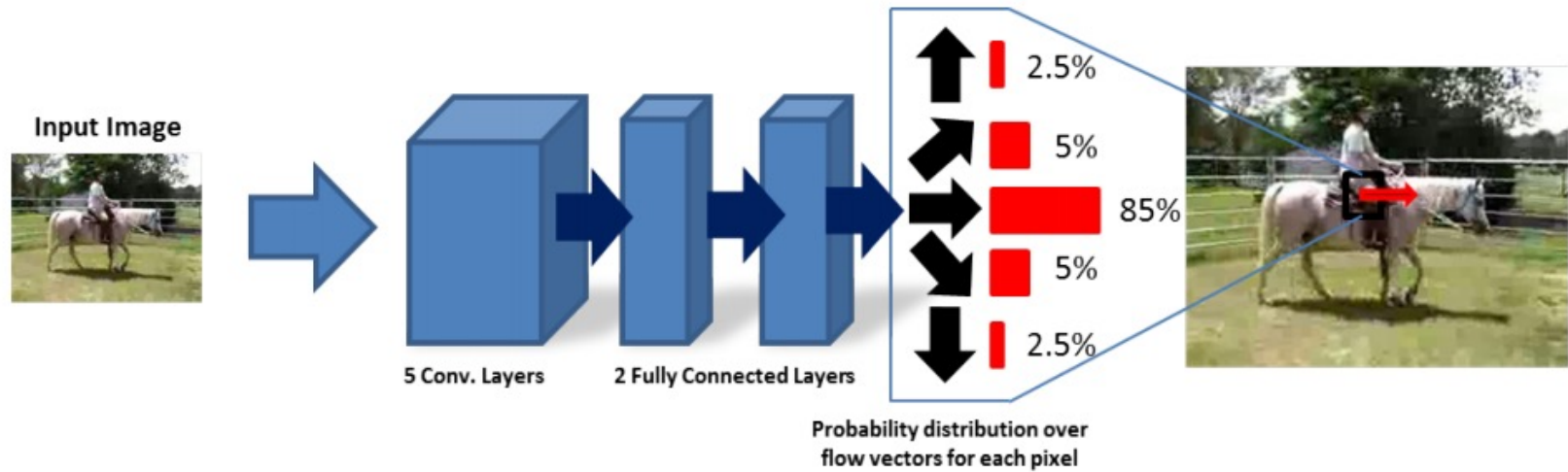
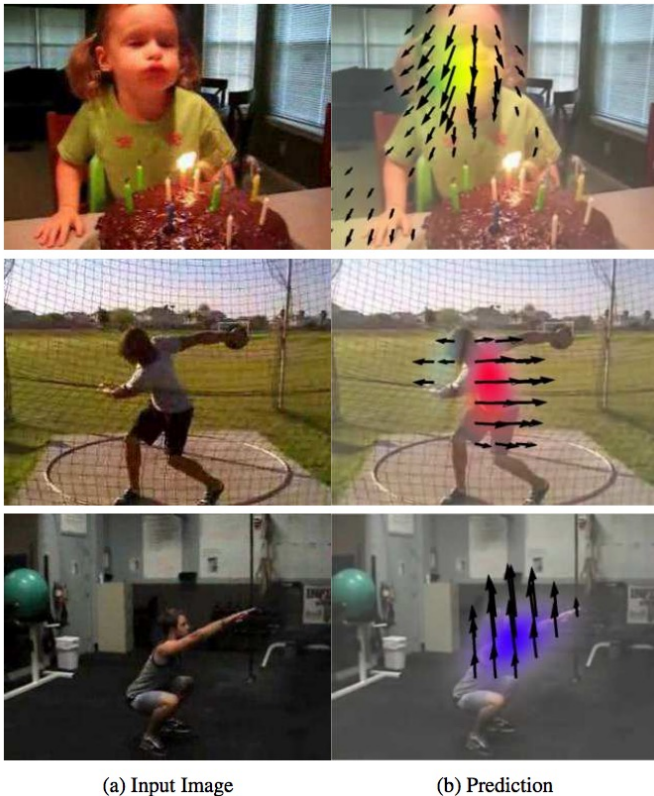
$$\begin{aligned} D \left(\begin{array}{c} \text{Query} \\ \text{Tracked} \end{array} \right) &< D \left(\begin{array}{c} \text{Query} \\ \text{Negative} \end{array} \right) \\ D \left(\begin{array}{c} \text{Query} \\ \text{Tracked} \end{array} \right) &< D \left(\begin{array}{c} \text{Query} \\ \text{Negative} \end{array} \right) \end{aligned}$$

D : Distance in deep feature space

(c) Ranking Objective

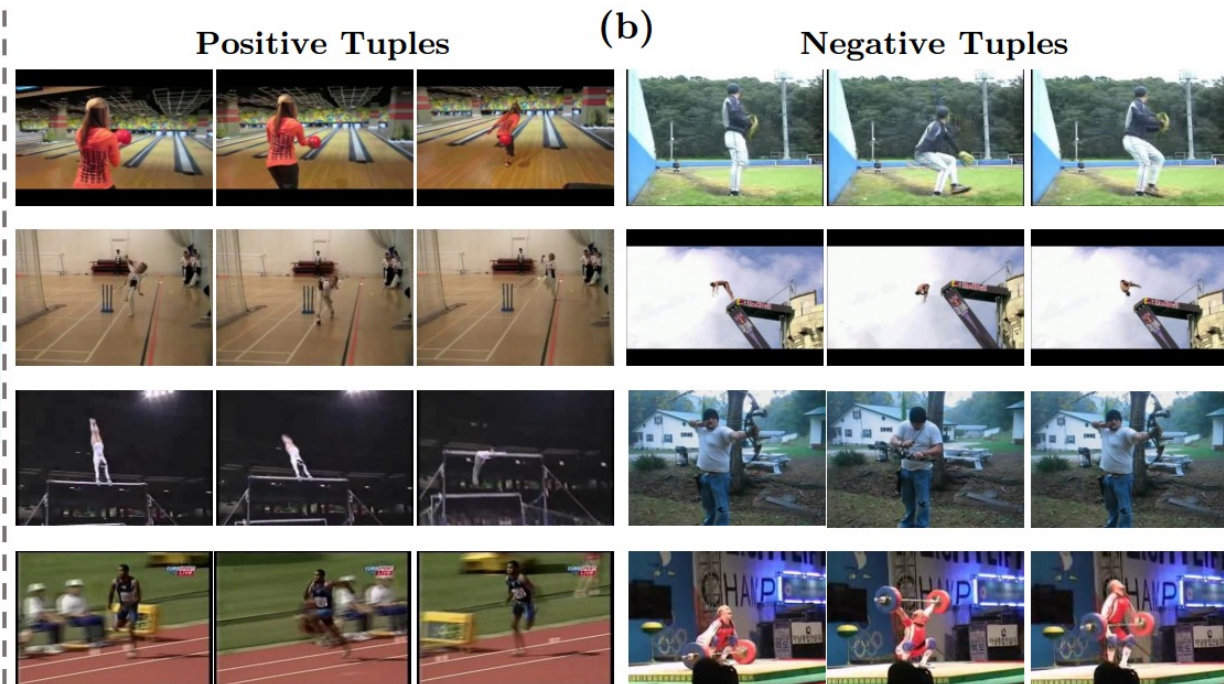
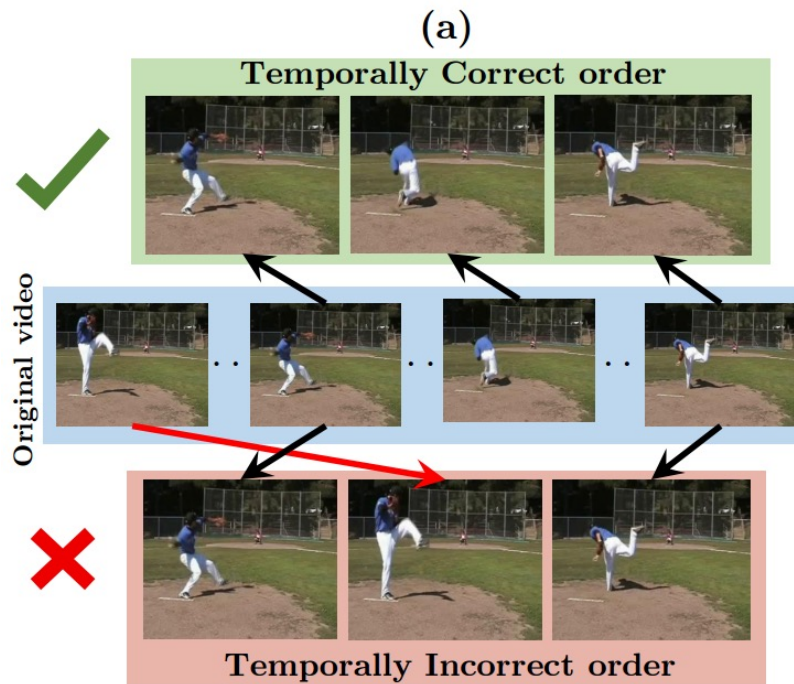
Exploiting motion

- Given a scene, what is going to move, and in what direction will it move?
- Learn common motion of visually similar objects / context



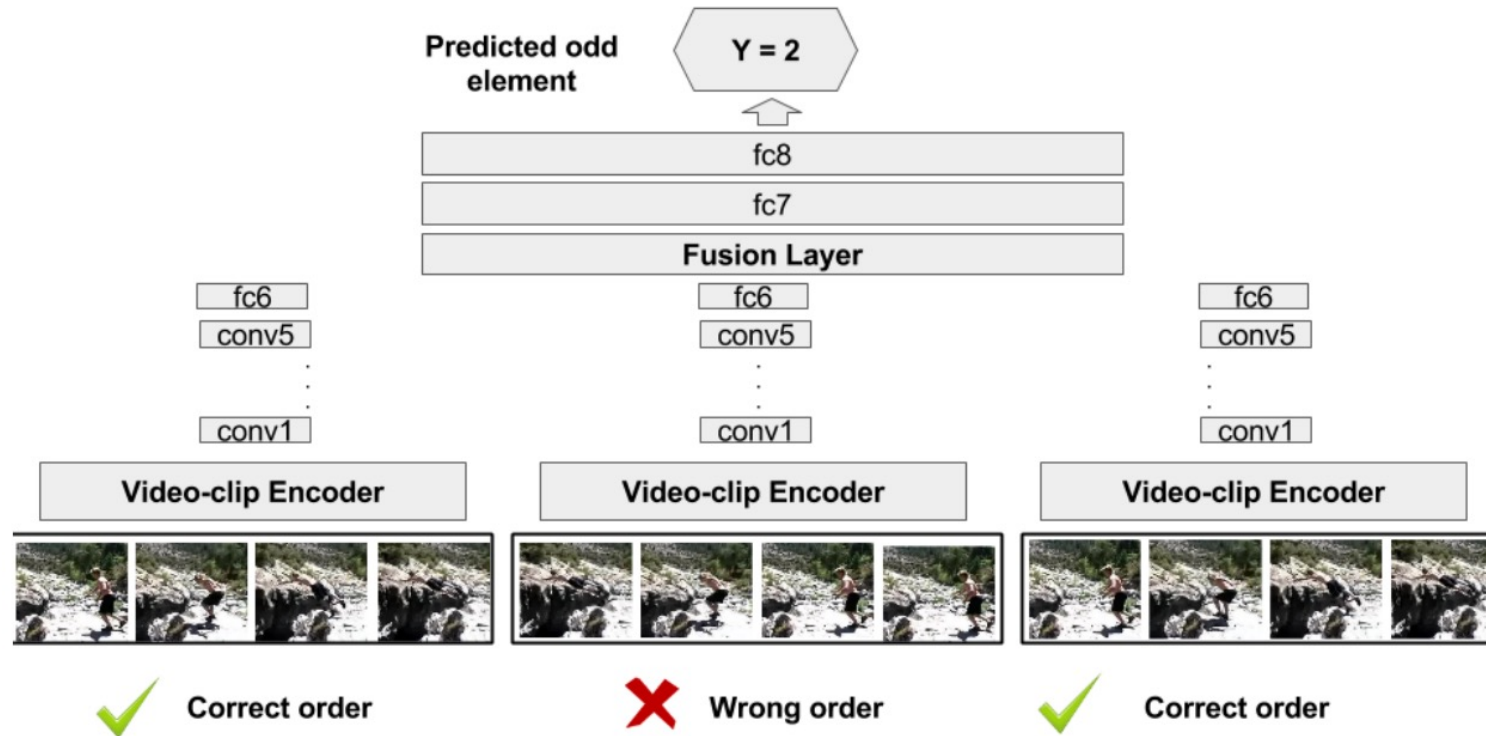
Order of actions/events

- Is the temporal order of a video correct?
 - Encode the cause and effect of action



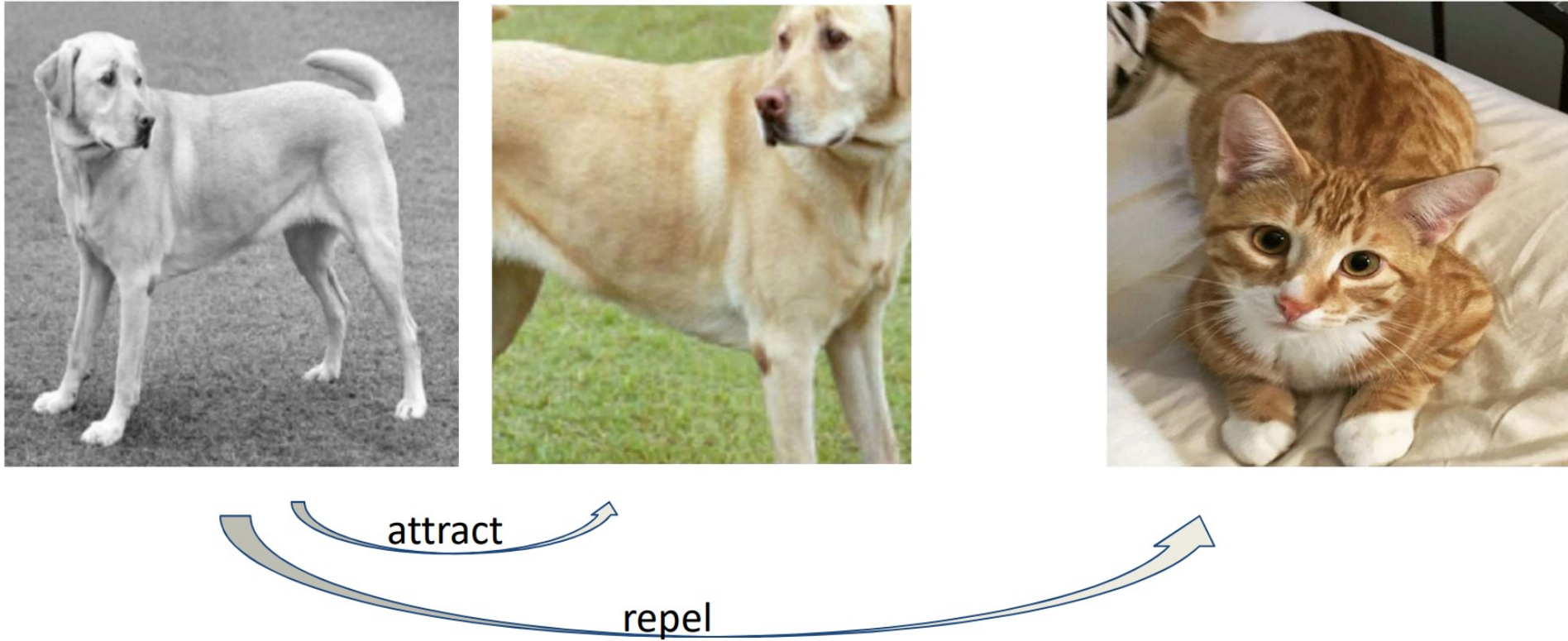
Order of actions/events

- Is the temporal order of a video correct?
 - Spotting the odd one



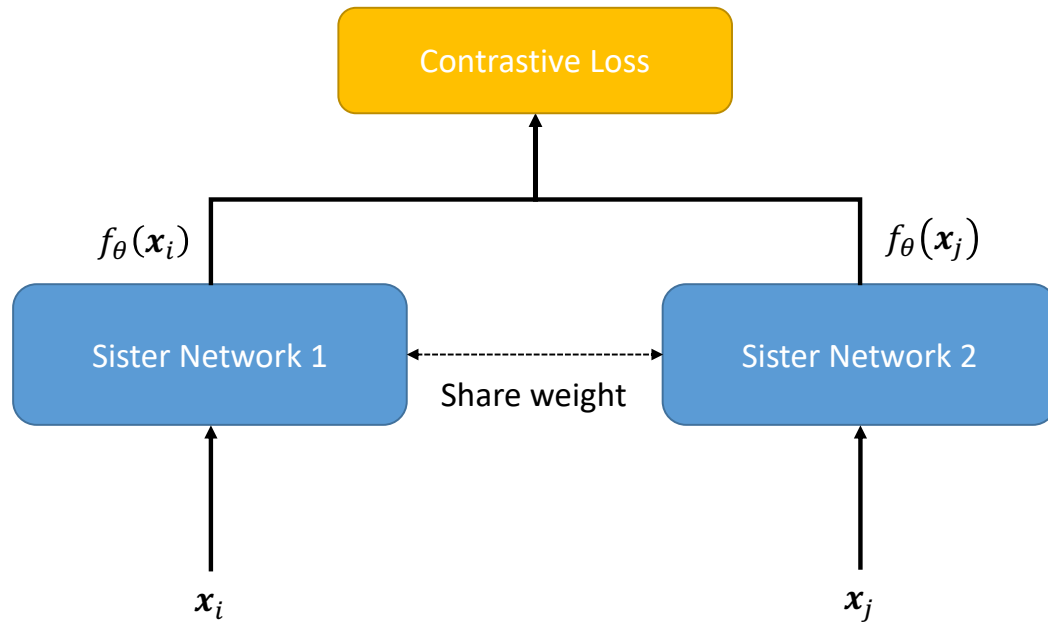
Contrastive Learning-based Methods

Contrastive learning



- Goal: learn an embedding space in which similar sample pairs stay close to each other while dissimilar ones are far apart
- Can be used in supervised or unsupervised learning

Contrastive loss



Siamese networks are weight-sharing neural networks applied on two or more inputs.

Natural tools for comparing for contrasting.

Siamese networks naturally introduce inductive biases for modeling invariance, as by definition “invariance” means that two observations of the same concept should produce the same outputs.

Contrastive loss

Given input samples $\{x_i\}$, each has a corresponding label $y_i \in \{1, \dots, L\}$ among L classes. **Contrastive loss** takes a pair of inputs (x_i, x_j)

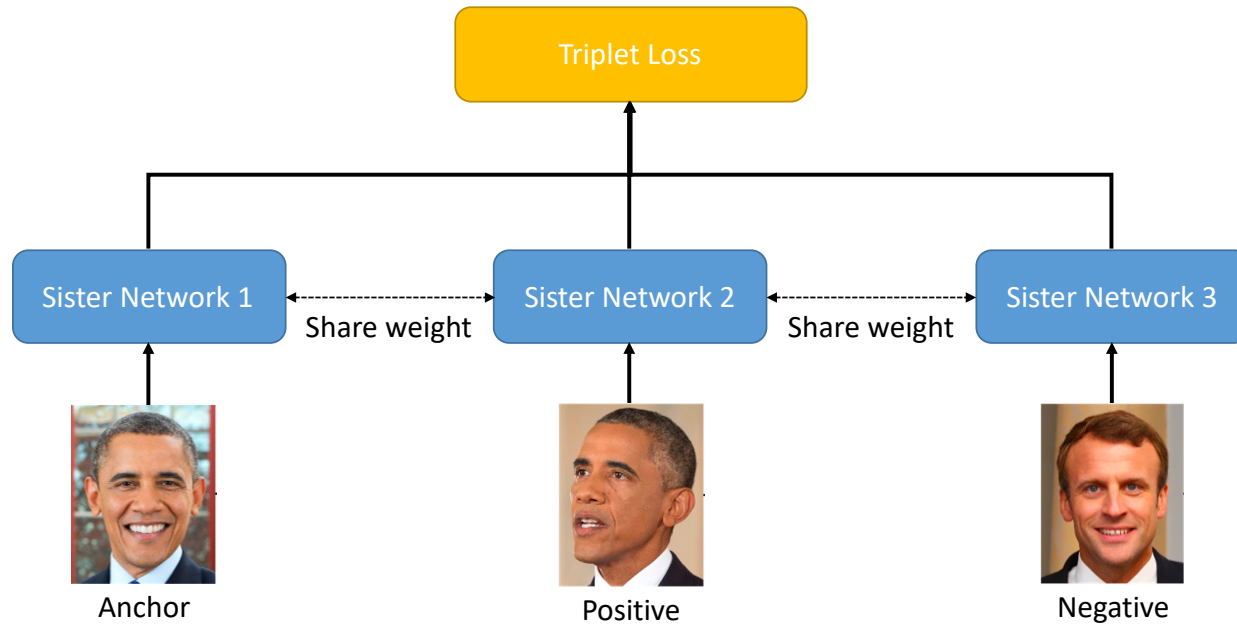
$$\mathcal{L}_{\text{cont}}(x_i, x_j) = \mathbf{1}_{[y_i=y_j]} \|f(x_i) - f(x_j)\|_2^2 + \mathbf{1}_{[y_i \neq y_j]} \max(0, \alpha - \|f(x_i) - f(x_j)\|_2)^2$$

minimizes the embedding distance
when they are from the **same** class

maximizes the embedding distance when
they are from the **different** class

α is a hyperparameter, defining the lower bound distance between samples of different classes - the distance $\|f(x_i) - f(x_j)\|_2^2$ must be larger or equal to α , else there will be a loss

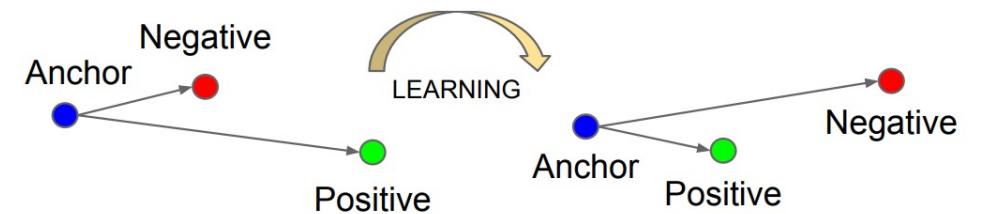
Triplet loss



Triplet loss was originally proposed in FaceNet and was used to learn face recognition of the same person at different poses and angles.

Mechanism

- Get anchor and positive from the same class
- Sample negative from a different class
- Minimizes the distance between an **anchor** and a **positive**, and
- Maximizes the distance between the **anchor** and a **negative**



Triplet loss



Anchor



Positive



Anchor



Negative

Triplet loss



Anchor



Positive



Anchor



Negative

Want: $\|f(A) - f(P)\|_2^2 \leq \|f(A) - f(N)\|_2^2$

$$\|f(A) - f(P)\|_2^2 - \|f(A) - f(N)\|_2^2 \leq 0$$

Triplet loss



Anchor



Positive



Anchor



Negative

Want:

$$\|f(A) - f(P)\|_2^2 \leq \|f(A) - f(N)\|_2^2$$

$$\underbrace{\|f(A) - f(P)\|_2^2}_0 - \underbrace{\|f(A) - f(N)\|_2^2}_0 \leq \underbrace{0}_{\text{Risk of trivial solution}}$$

Triplet loss



Anchor



Positive



Anchor



Negative

Want:

$$\|f(A) - f(P)\|_2^2 \leq \|f(A) - f(N)\|_2^2$$

$$\|f(A) - f(P)\|_2^2 - \|f(A) - f(N)\|_2^2 \leq 0 - \alpha \quad \text{Prevent trivial solution}$$

Triplet loss



Anchor



Positive



Anchor



Negative

Want:

$$\|f(A) - f(P)\|_2^2 + \alpha \leq \|f(A) - f(N)\|_2^2$$

$$\|f(A) - f(P)\|_2^2 - \|f(A) - f(N)\|_2^2 + \alpha \leq 0$$

Margin

Triplet loss



Anchor



Positive

$$d(A, P) = 0.5 \quad \alpha = 0.2$$

Want:

$$\|f(A) - f(P)\|_2^2 + \alpha \leq$$

$$\|f(A) - f(P)\|_2^2 - \|f(A) - f(N)\|_2^2 + \alpha \leq 0$$



Anchor



Negative

$$d(A, N) = 0.51$$

$$\|f(A) - f(N)\|_2^2$$

$d(A, N) = 0.51$ is not good enough!

We want it to be at least 0.7 or higher.

Either push $d(A, N)$ up or $d(A, P)$ down

Triplet loss



Anchor



Positive



Anchor



Negative

Want:

$$\|f(A) - f(P)\|_2^2 + \alpha \leq \|f(A) - f(N)\|_2^2$$
$$\|f(A) - f(P)\|_2^2 - \|f(A) - f(N)\|_2^2 + \alpha \leq 0$$

Margin helps to learn a better embedding by pushing the distance of anchor-negative away and reduce the distance between anchor-positive

Triplet loss

Given 3 images: A, P, N

$$\mathcal{L}_{\text{triplet}}(A, P, N) = \max(\|f(A) - f(P)\|_2^2 - \|f(A) - f(N)\|_2^2 + \alpha, 0)$$

If this term is ≤ 0 then $L(A, P, N)$ would take the value of zero. Otherwise, there will be a positive loss

Loss over a set of triplets

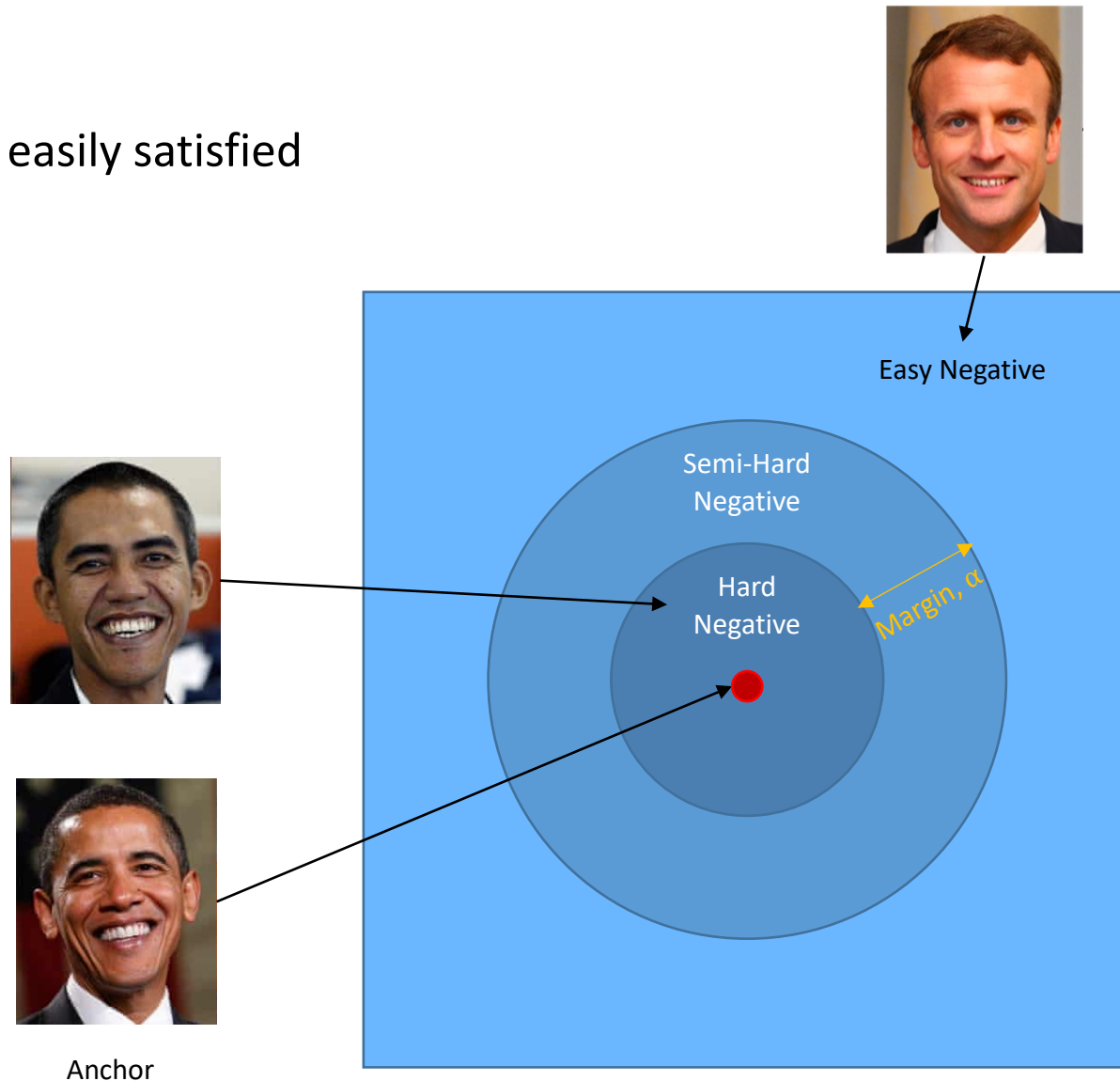
$$J = \sum_{x \in \mathcal{X}} \mathcal{L}_{\text{triplet}}(A_i, P_i, N_i)$$

Training set : 10K images of 1K classes

Select A, P, N

Choosing the triplet A, P, N

- During training, if you choose A, P, N randomly
 $\|f(A) - f(P)\|_2^2 - \|f(A) - f(N)\|_2^2 + \alpha \leq 0$ is easily satisfied
- Choose triplets that are “hard” to train on
 - Hard negatives – negative samples that look like positive
 - Semi-hard negatives – lie inside α .
Selecting the hardest negatives can in practice lead to bad local minima early on in training
- Generate triplets online. This can be done by selecting the hard positive/negative exemplars from within a mini-batch.



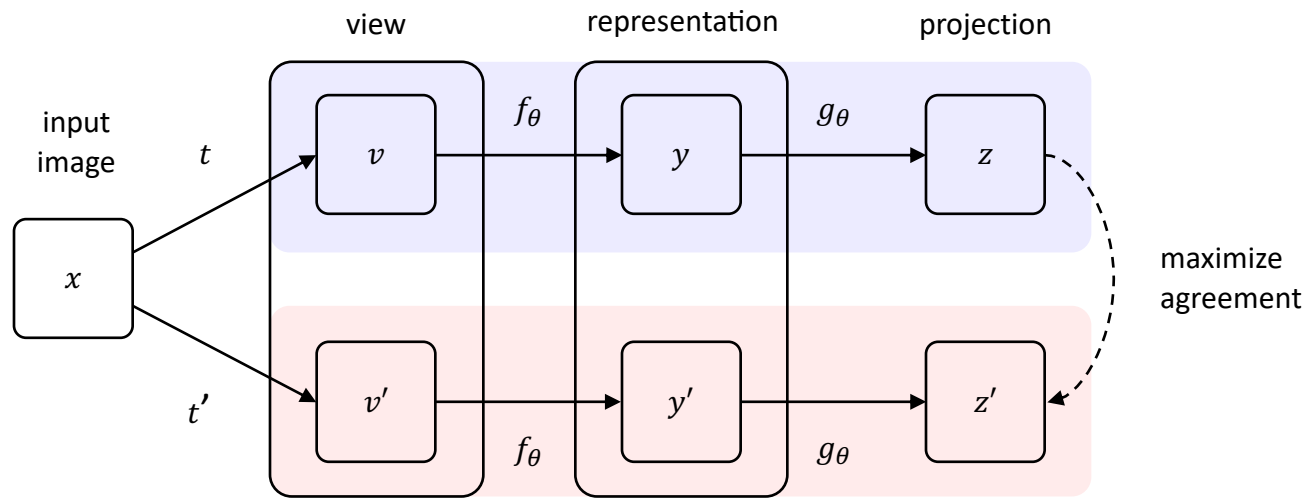
InfoNCE

The InfoNCE contrastive loss with temperature τ is used over one positive P and multiple negative samples

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(A \cdot P / \tau)}{\exp(A \cdot P / \tau) + \sum_{i=0} \exp(A \cdot N_i / \tau)}$$

Intuitively, this loss is the log loss of a $(|N_i| + 1)$ -way softmax-based classifier that tries to classify A as P

SimCLR



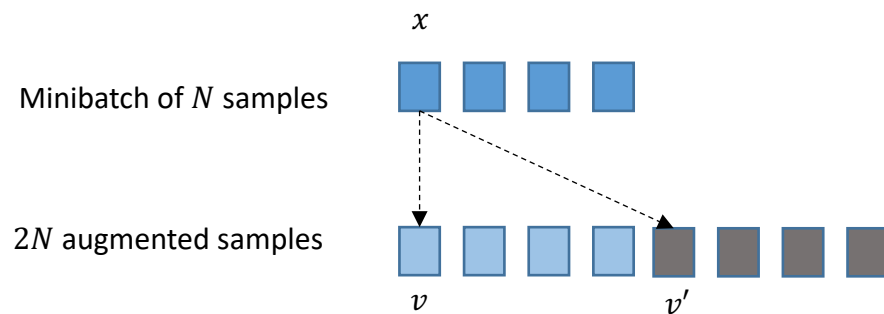
Learns representations for visual inputs by **maximizing agreement** between differently **augmented views** of the same sample via a **contrastive loss** in the latent space

1) Randomly sample a minibatch of N samples and each sample is applied with two different data augmentation operations, resulting in $2N$ augmented samples in total.

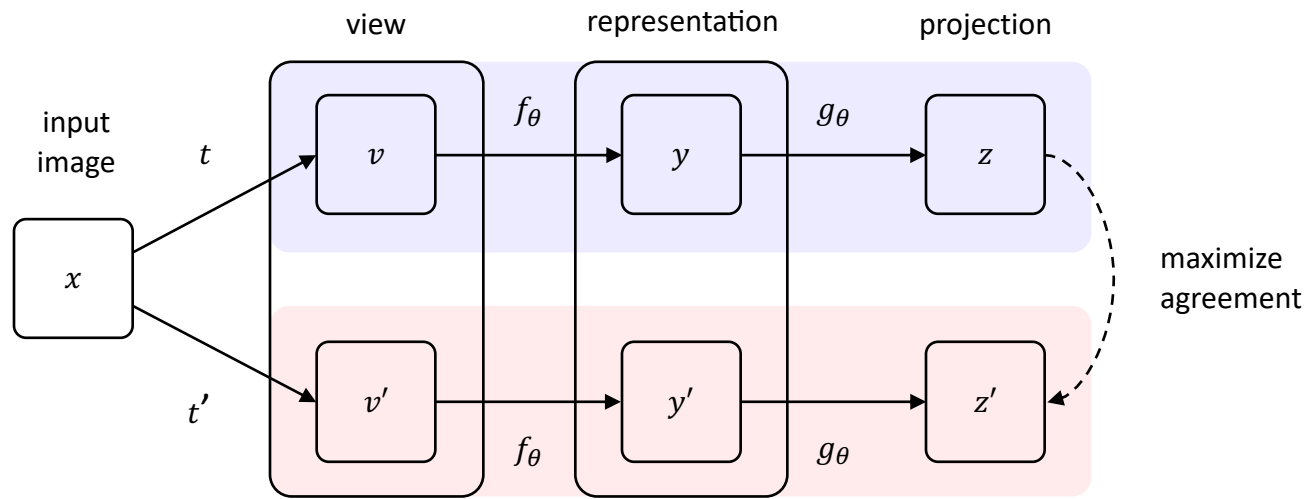
$$v = t(x) \text{ and } v' = t'(x), t, t' \sim \mathcal{T}$$

where two separate data augmentation operators, t and t' , are sampled from the same family of augmentations $\mathcal{T}$.

Data augmentation includes random crop, resize with random flip, color distortions, and Gaussian blur.



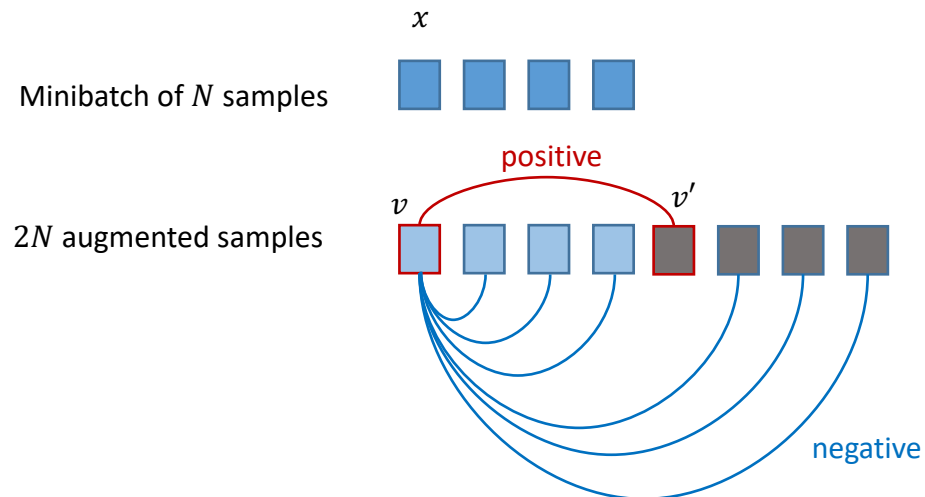
SimCLR



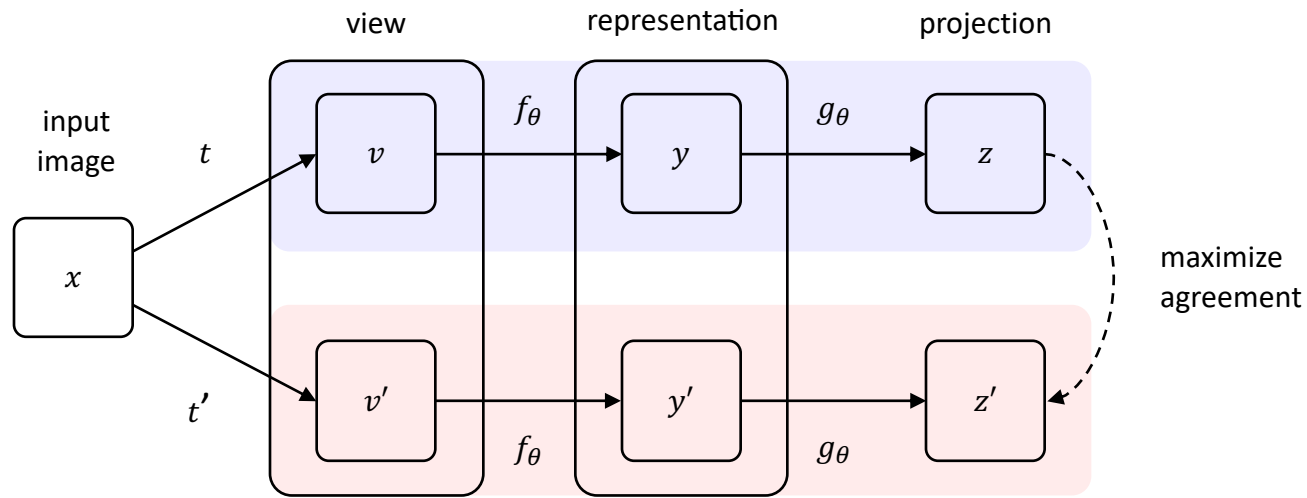
2) Given one positive pair, other $2(N - 1)$ data points are treated as negative samples.

The representation is produced by a base encoder $f_\theta(\cdot)$:

$$y = f_\theta(v), y' = f_\theta(v')$$



SimCLR



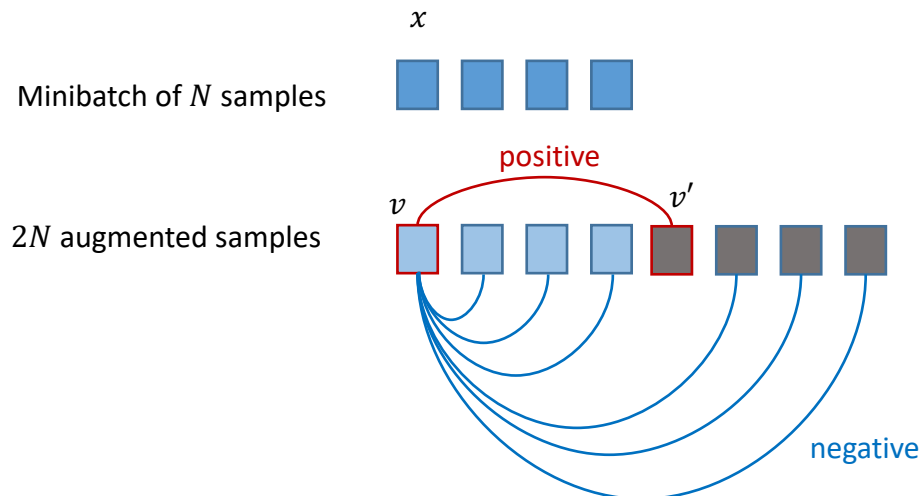
3) The contrastive learning loss is defined using dot product between L2-normalized $\mathbf{u}$ and $\mathbf{v}$ (i.e., cosine similarity)

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \mathbf{u} \cdot \mathbf{v} / \|\mathbf{u}\| \|\mathbf{v}\|$$

The loss operates on an extra projection layer of the representation $g_\theta(\cdot)$ rather than on the representation space directly.

But only $f_\theta(\cdot)$ is used for downstream tasks

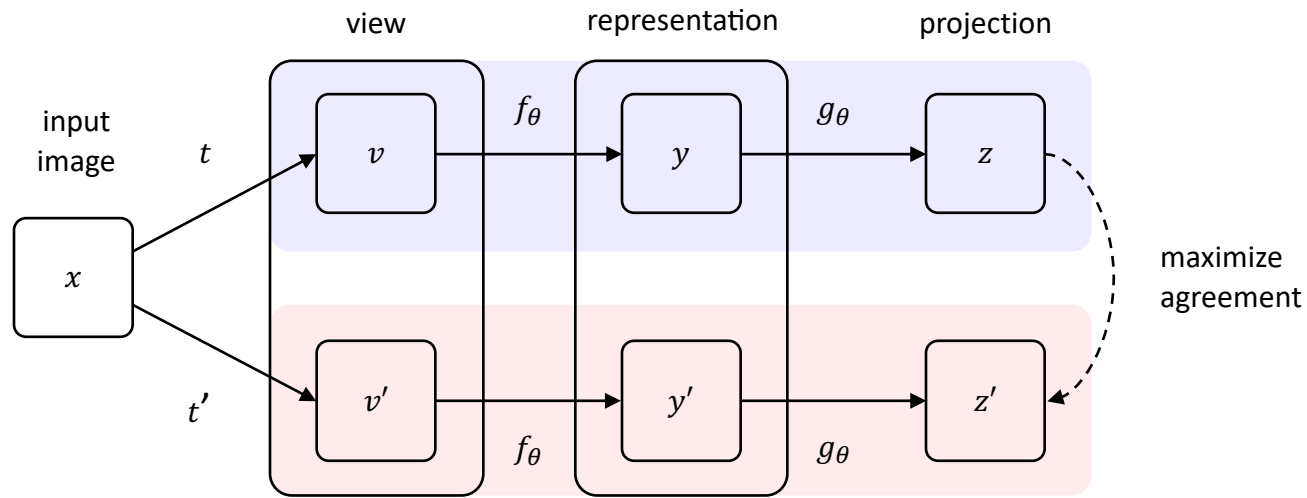
$$z = g_\theta(y) \text{ and } z' = g_\theta(y')$$



$$\mathcal{L}_{\text{SimCLR}} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)}$$

- An appropriate temperature parameter can help the model learn from hard negatives.
- Optimal temperature differs on different batch sizes and number of training epochs.

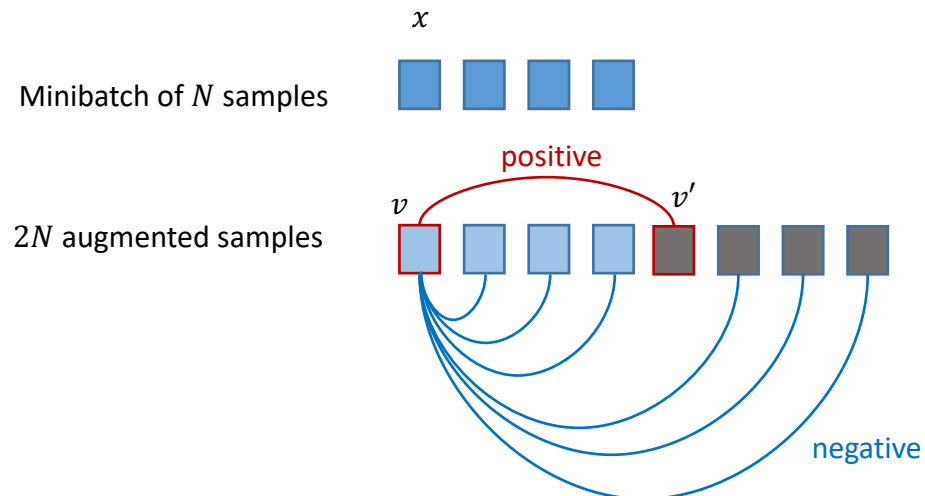
SimCLR



SimCLR needs a **large batch size** (e.g., 4096) to incorporate enough negative samples to achieve good performance.

Only when the batch size is big enough, the loss function can cover a **diverse** enough collection of negative samples, challenging enough for the model to learn meaningful representation to distinguish different examples.

Computing embeddings for a large number of negative samples in every batch is extremely **expensive**!

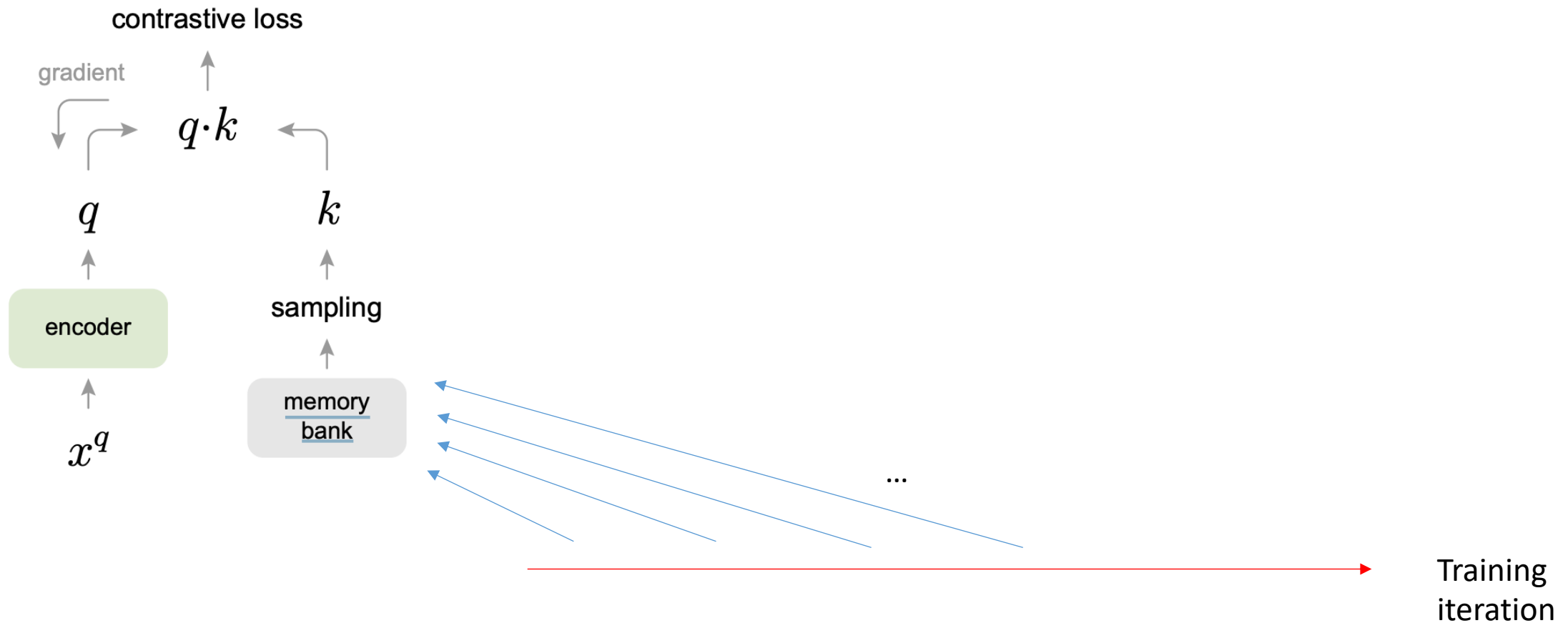


Solution: Memory bank

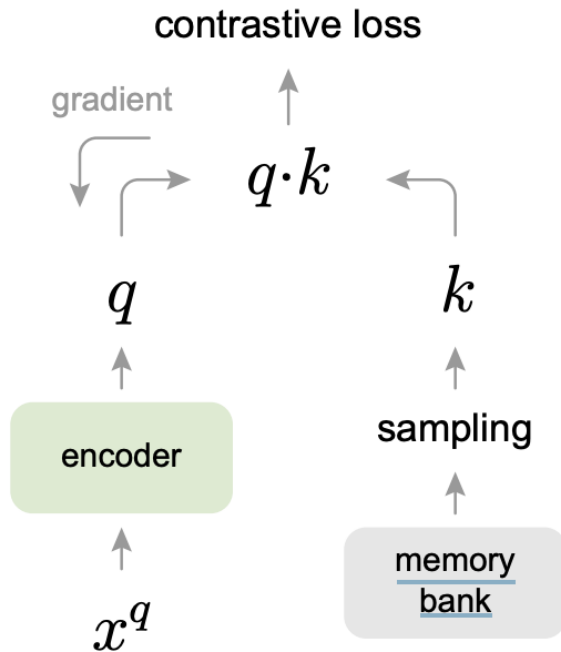
Storing sample representation in the database from past iterations!

Memory bank

Storing sample representation in the database from past iterations!



Memory bank



A **naïve memory bank** consists of the representations of **all past samples** in the dataset.

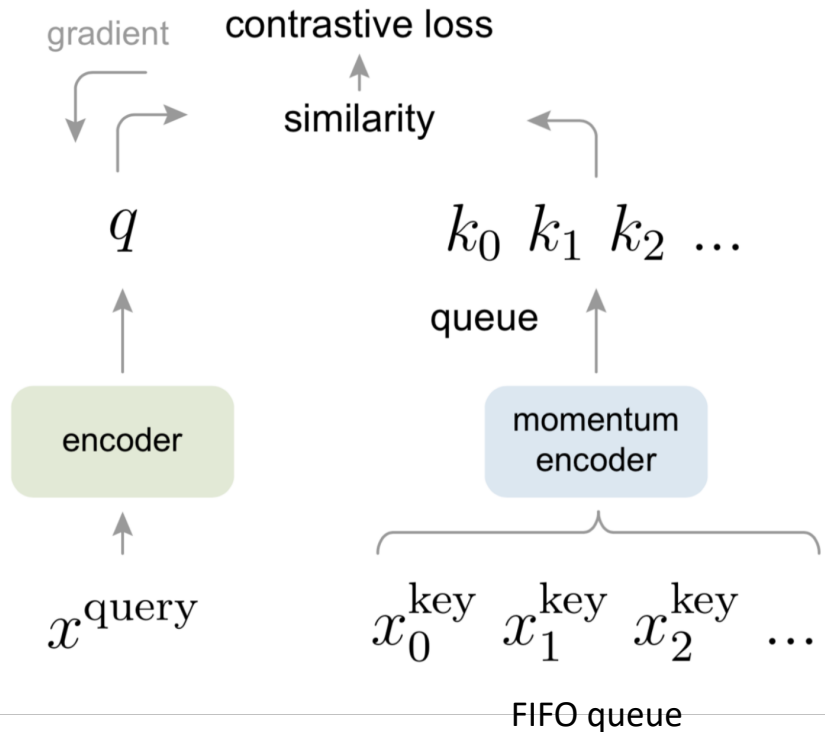
Good: Can support a large dictionary

Bad: Data **staleness**

- The representation of the memory bank was updated when it was last seen
- So the sampled keys are essentially about the encoders at multiple different steps all over the past epoch and thus are less consistent.

How to solve this problem?

Momentum contrast (MoCo)



Memory bank as a large FIFO queue

Reuse the encoded keys from the **immediate preceding mini-batches**. The size can be much larger than a typical mini-batch size

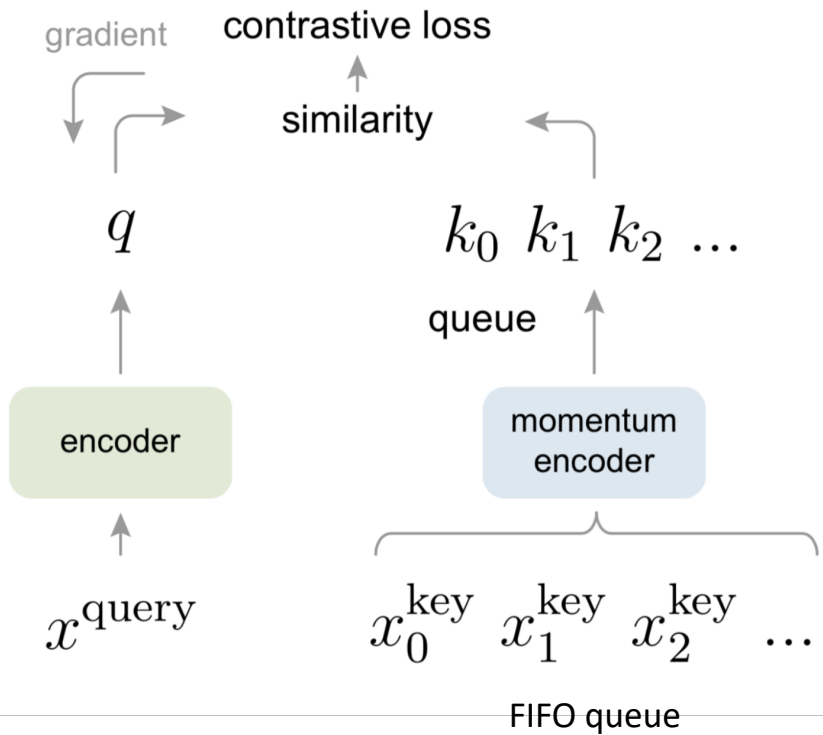
Momentum encoder

To address data staleness – the encoder for the dictionary keys is kept **as consistent as possible** despite its evolution

$$\theta_k = m\theta_k + (1 - m)\theta_q$$

The parameters of key encoder evolves more **smoothly** than the query encoder

Momentum contrast (MoCo)



Encoded query should be **similar to its matching key** and **dissimilar to others**.

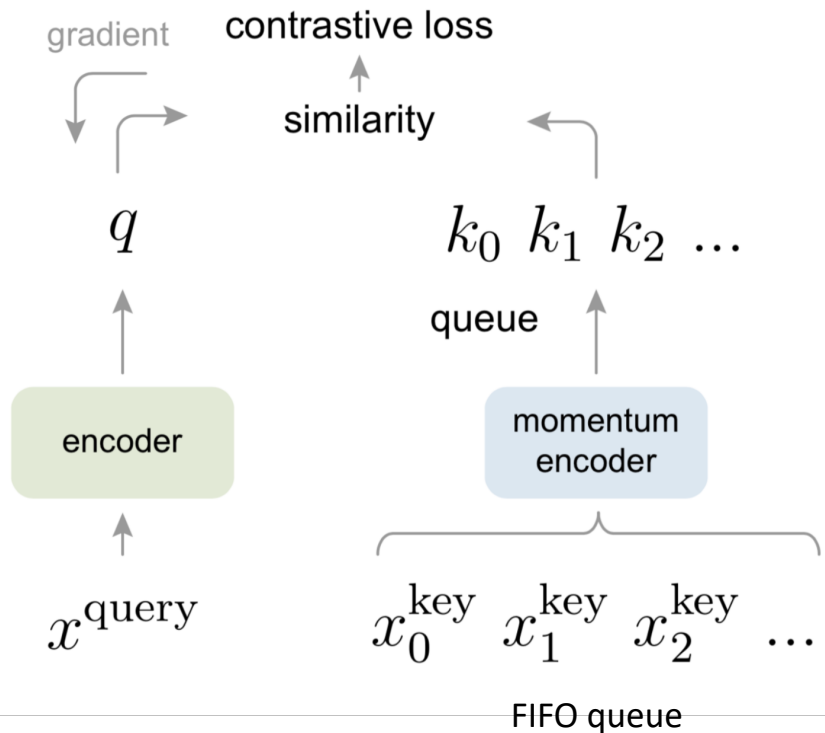
Given a query sample x^{query} , we get a query representation through an encoder $q = f_q(x^{\text{query}})$.

A list of key representations $\{k_0, k_1, \dots\}$ in the dictionary are encoded by a momentum encoder $k_i = f_k(x_i^{\text{key}})$.

Let's assume among them there is a single *positive* key k^+ in the dictionary that matches q .

The positive key k^+ is created using a noisy copy of x^{query} with different augmentation.

Momentum contrast (MoCo)



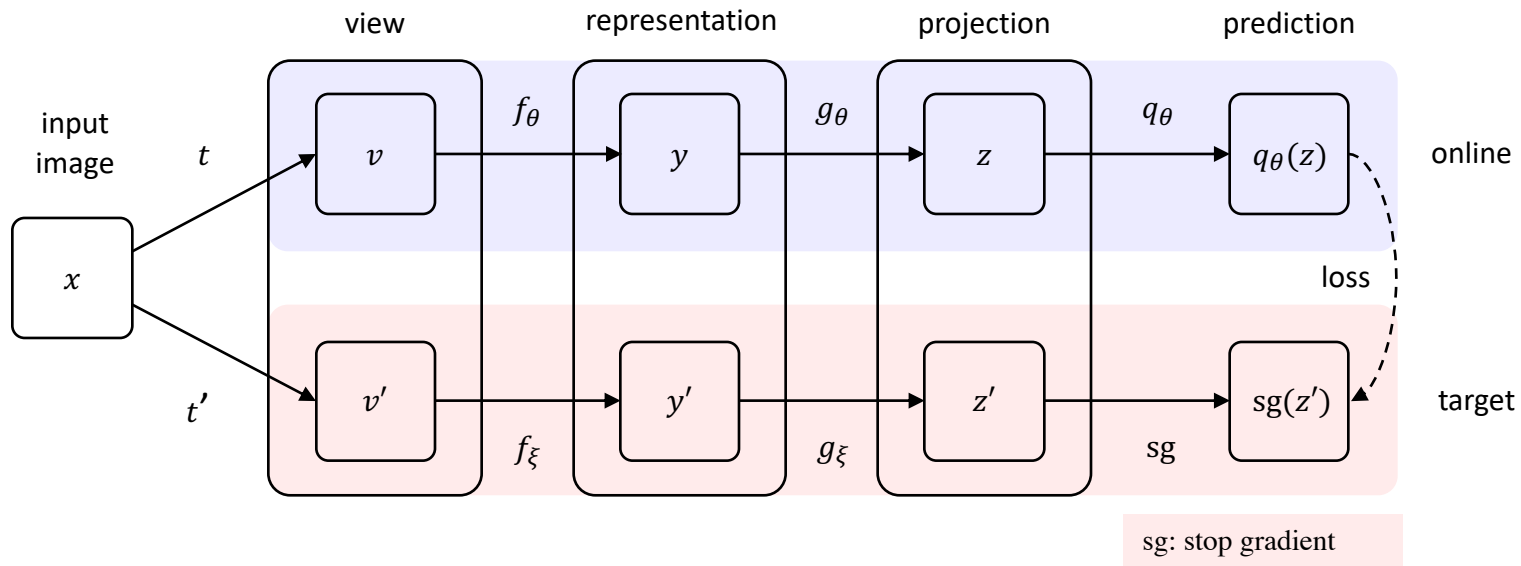
Then the InfoNCE contrastive loss with temperature τ is used over one positive and negative samples

$$\mathcal{L}_{\text{MoCo}} = -\log \frac{\exp(q \cdot k^+ / \tau)}{\sum_{i=0}^K \exp(q \cdot k_i / \tau)}$$

Intuitively, this loss is the log loss of a $(K + 1)$ -way softmax-based classifier that tries to classify q as k^+

Can we learn *without negative samples*? Potentially remove the momentum encoder.

BYOL



Learn without negative samples

Online network (parameterized by θ)

- An encoder f_θ ;
- A projector g_θ ;
- A predictor q_θ .

Target network (momentum encoder parameterized by ξ)

Same network architecture but with polyak averaged weights,

$$\xi \leftarrow \tau \xi + (1 - \tau) \theta$$

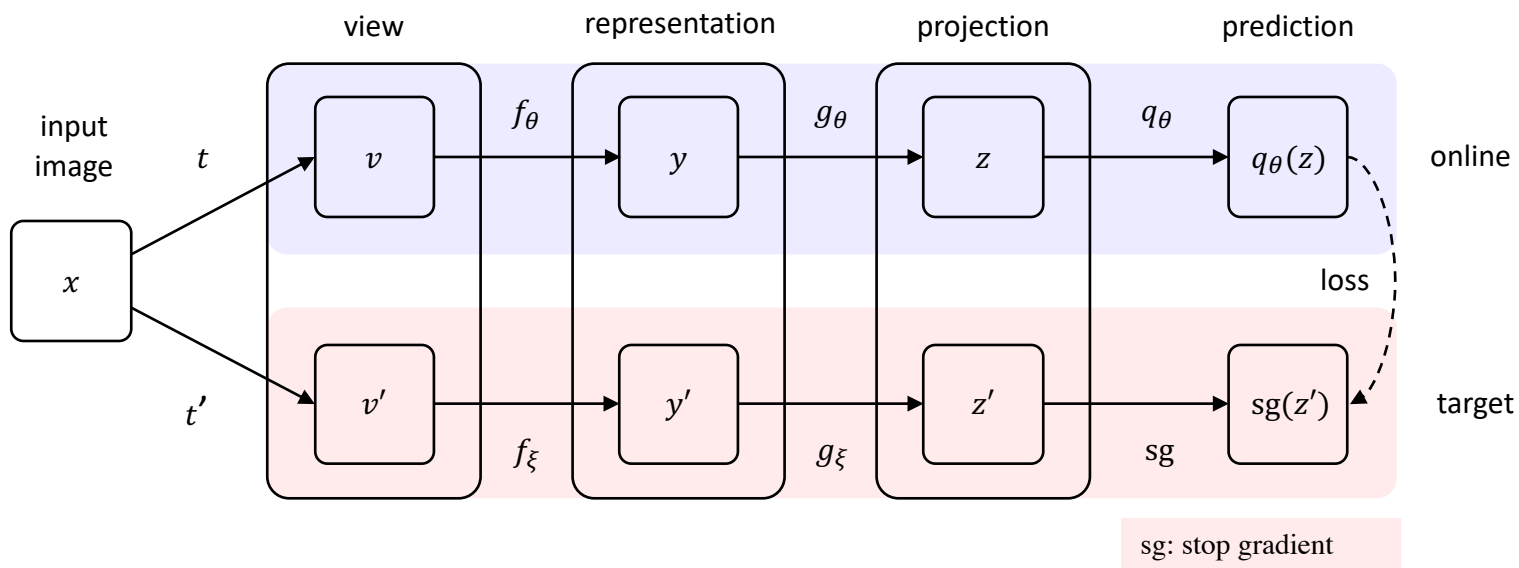
The **goal** is to learn $f_\theta(\cdot)$ that can be used in downstream tasks.

Polyak averaging: sets final parameters to an average of (recent) parameters visited in the optimization trajectory

Jean-Bastien Grill et al., [Bootstrap your own latent: A new approach to self-supervised Learning](#), NeurIPS 2020

BYOL

Given an image x , the BYOL loss is constructed as follows:



1. Create two augmented views:

$$v = t(x) \text{ and } v' = t'(x)$$

2. They are encoded into representations,

$$y = f_\theta(v) \text{ and } y' = f_\xi(v')$$

3. They are projected into latent variables,

$$z = g_\theta(y) \text{ and } z' = g_\xi(y')$$

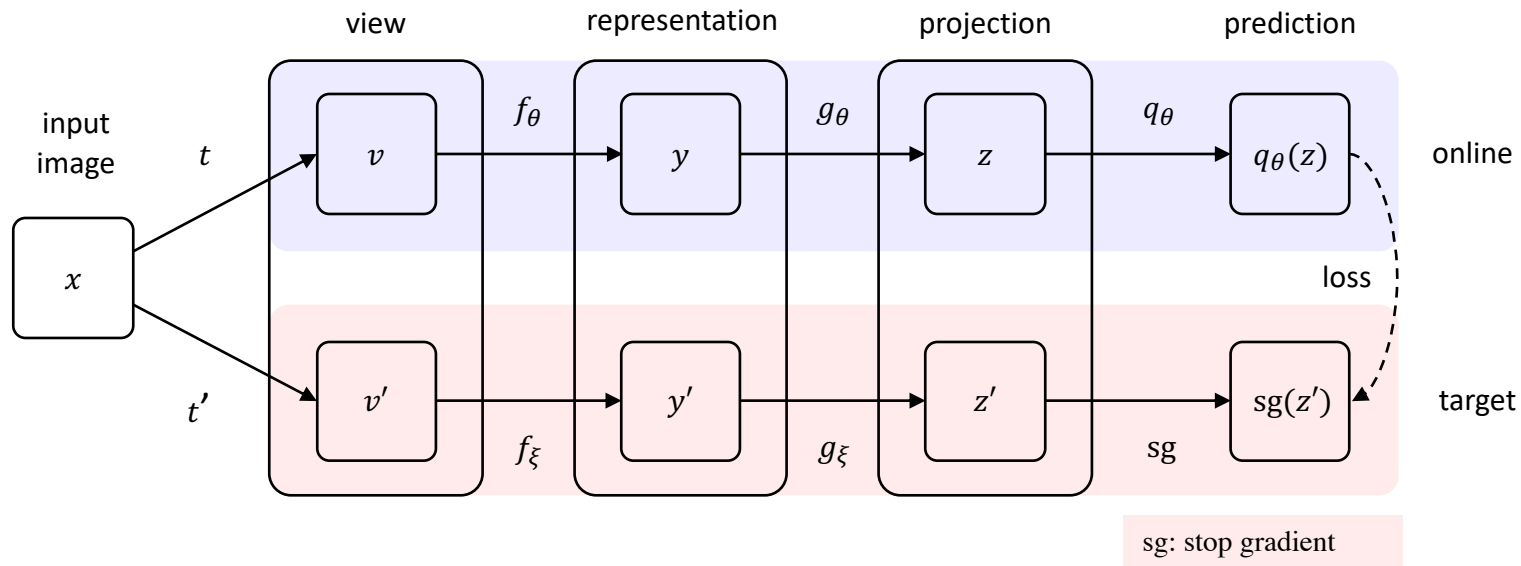
4. Online network outputs a prediction $q_\theta(z)$

5. Both $q_\theta(z)$ and z' are L2-normalized, giving us

$$\bar{q}_\theta(z) = q_\theta(z) / \|q_\theta(z)\| \text{ and } \bar{z}' = z' / \|z'\|$$

BYOL

Given an image x , the BYOL loss is constructed as follows:



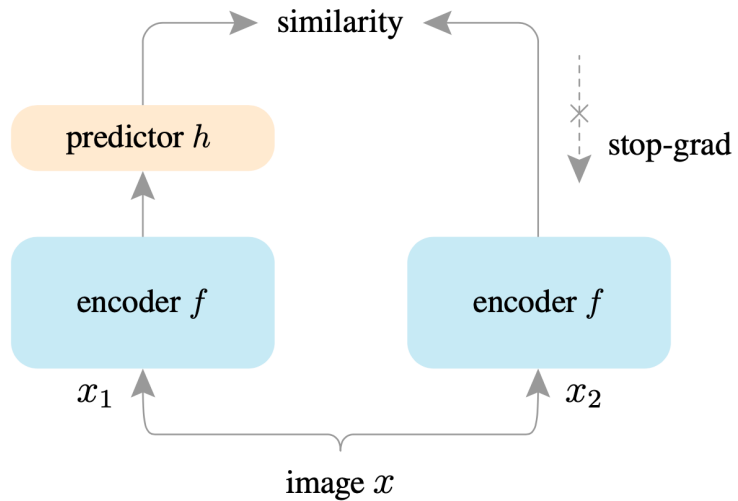
6. The loss $\mathcal{L}_{\text{BYOL}}^\theta$ is MSE between L2-normalized prediction $\bar{q}_\theta(z)$ and $\bar{z}'$

7. The other symmetric loss $\tilde{\mathcal{L}}_{\text{BYOL}}^\theta$ can be generated by switching v' and v ; that is, feeding v' to online network and v to target network

8. The final loss is $\mathcal{L}_{\text{BYOL}}^\theta + \tilde{\mathcal{L}}_{\text{BYOL}}^\theta$ and only parameters θ are optimized (stop gradient in ξ)

*What is the role of **stop gradient**?*

SimSiam



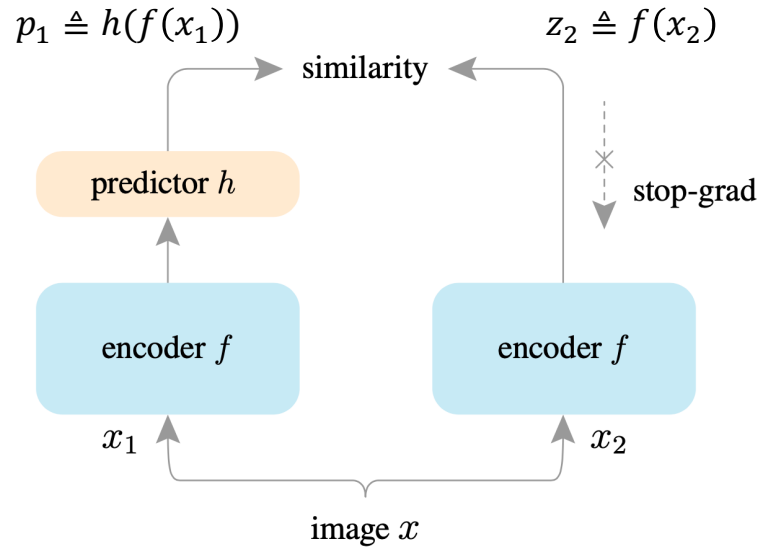
Stop gradient is the key to prevent trivial solution that collapses to a constant!

No momentum encoder is needed, unlike BYOL or MoCo

No negative samples as needed in MoCo and SimCLR

Not requiring large batch size

SimSiam



Steps

- 1) Takes as input two randomly augmented views x_1 and x_2 from an image x .
- 2) The two views are processed by an encoder network f
- 3) A prediction MLP head, denoted as h , transforms the output of one view and matches it to the other view
- 4) Minimize their negative cosine similarity

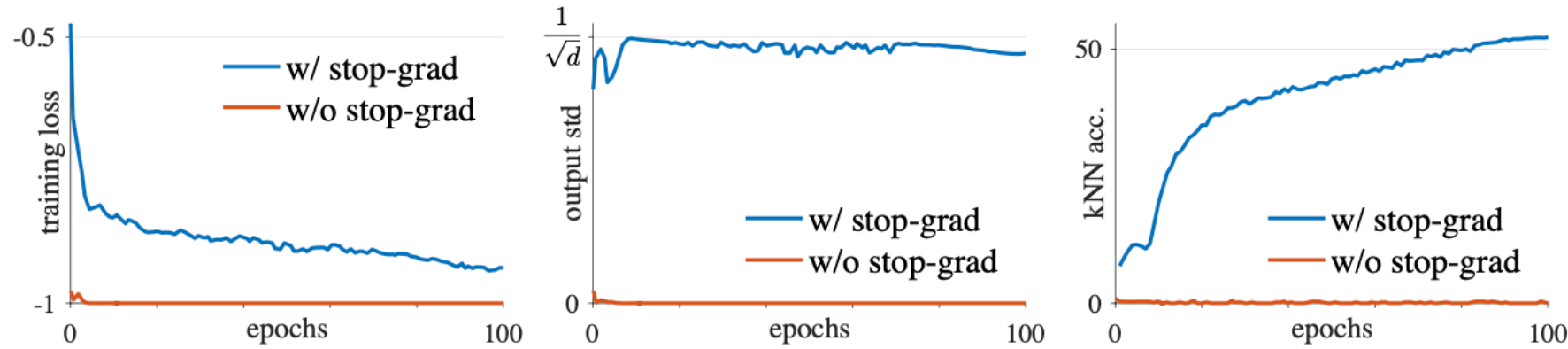
$$\mathcal{D}(p_1, z_2) = -\frac{p_1}{\|p_1\|_2} \cdot \frac{z_2}{\|z_2\|_2}$$

- 5) Stop gradient (sg)

$$\mathcal{D}(p_1, \text{sg}(z_2))$$

This means that z_2 is treated as a constant in this term. The encoder on x_2 receives no gradient from z_2

SimSiam



	acc. (%)
w/ stop-grad	67.7 ± 0.1
w/o stop-grad	0.1

Left plot: training loss. Without stop-gradient the learning degenerates immediately.

Middle plot: the per-channel std of the l2-normalized output, plotted as the averaged std over all channels.

Right plot: validation accuracy of a kNN classifier [34] as a monitor of progress.

Table: ImageNet linear evaluation

SimSiam

The role of stop gradient

Consider solving

$$\min_{\theta, \eta} \mathcal{L}(\theta, \eta)$$

We can solve this by an **Expectation-Maximization (EM)**, fixing one set of variables and solving for the other set

$$\begin{aligned}\theta^t &= \underset{\theta}{\operatorname{argmin}} \mathcal{L}(\theta, \eta^{t-1}), \\ \eta^t &= \underset{\eta}{\operatorname{argmin}} \mathcal{L}(\theta^t, \eta)\end{aligned}$$

Take k-means clustering as analogy

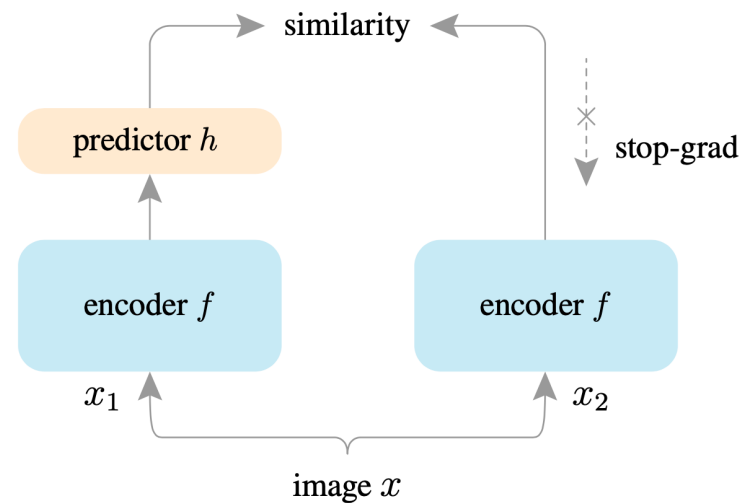
- Expectation step: *Finding the clustering centers*
- Maximization step: *Finding assignment vector of the sample x (a one-hot vector in k -means)*

SimSiam

The role of stop gradient

SimSiam is an implementation of an **Expectation-Maximization (EM)** like algorithm.

Second, obtaining θ^{t+1} , the parameters of the network

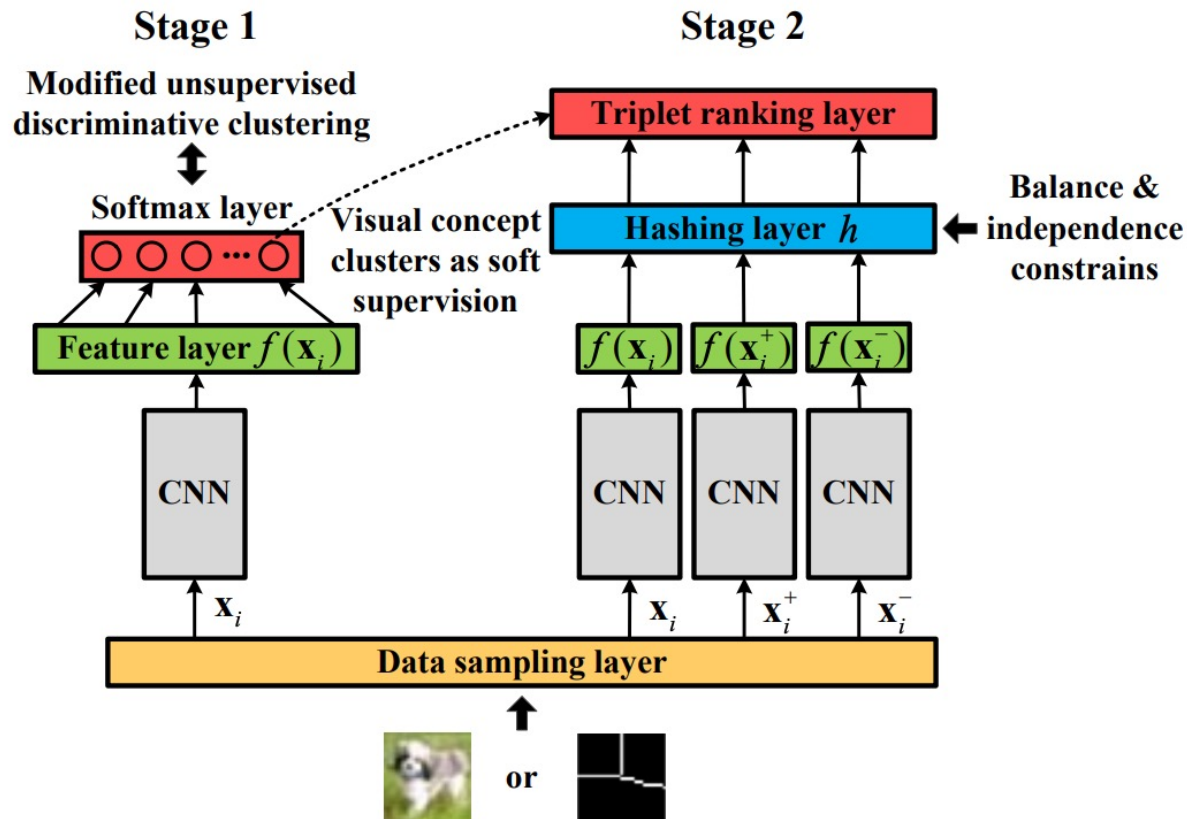


First, obtain η_x^t , the representation of the image x

The stop-gradient operation is a natural consequence, because the gradient does not back-propagate to η_x^t , which is a constant in this subproblem.

Clustering-based Methods

Two-stage method

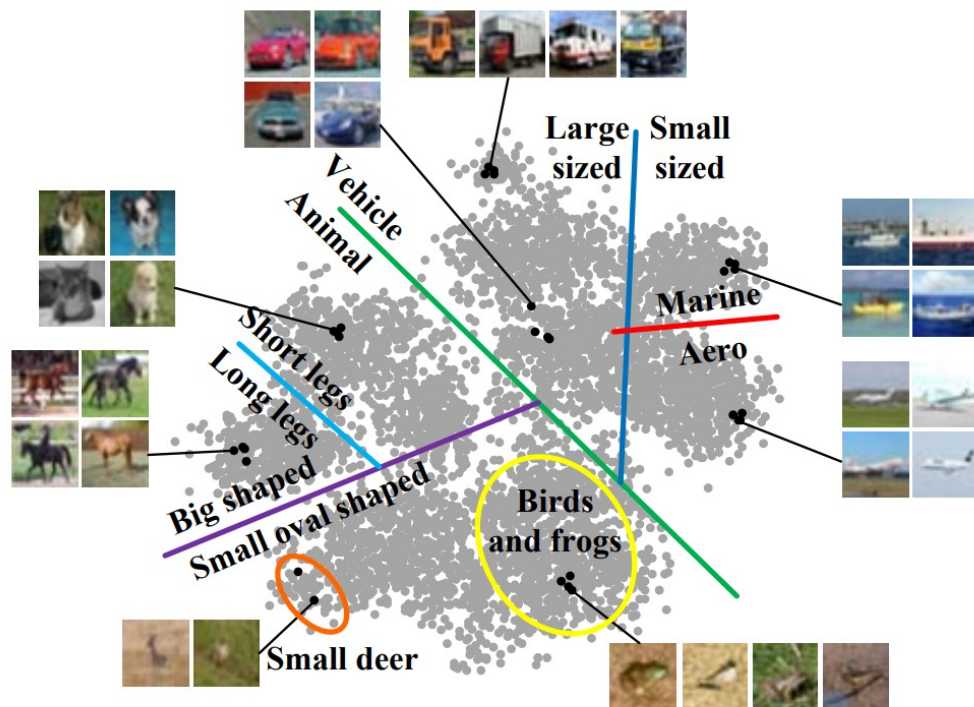


A two-stage pipeline for learning attributes as binary hash codes. All CNNs have shared architectures and weights

The first stage performs **alternating unsupervised discriminative clustering and representation learning with CNN** to discover representative **visual concepts**. The concepts can correspond to frequent image primitives or objects (segments)

They are passed to the next stage as a soft guidance. Specifically, we perform discriminative hashing under a triplet ranking loss to predict binary attributes, while fine-tuning the feature representations

Two-stage method

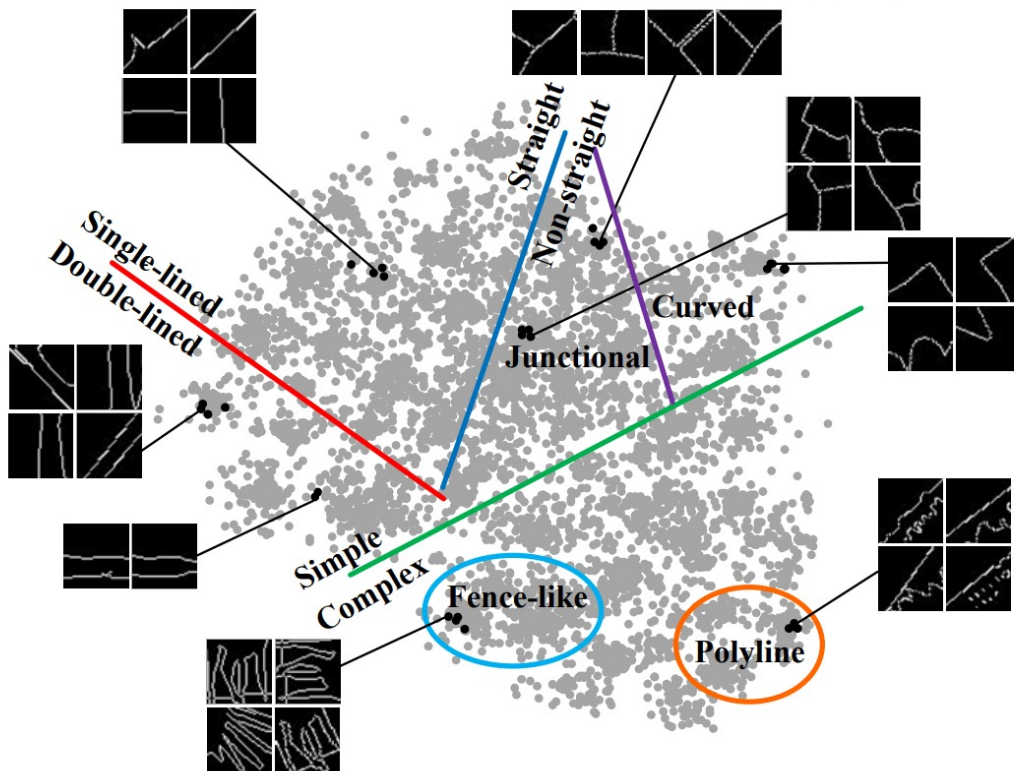


2D feature space of unsupervisedly learned attributes for natural images on CIFAR-10 and binary contour patches on BSDS500.

The colored lines delineate the approximate separation borderlines of the binary attributes, which are discriminative and easily interpreted semantically in both cases.

Many attributes are shared across categories, and they together can help distinguish the categories of interest.

Two-stage method

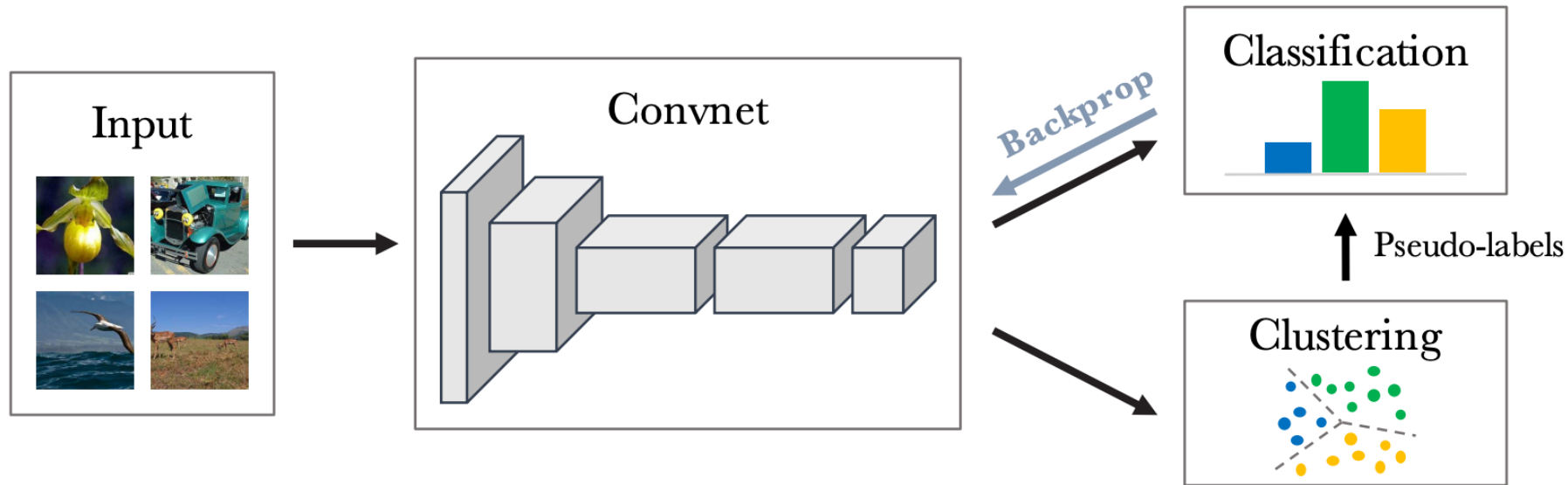


2D feature space of unsupervisedly learned attributes for natural images on CIFAR-10 and binary contour patches on BSDS500.

The colored lines delineate the approximate separation borderlines of the binary attributes, which are discriminative and easily interpreted semantically in both cases.

Many attributes are shared across categories, and they together can help distinguish the categories of interest.

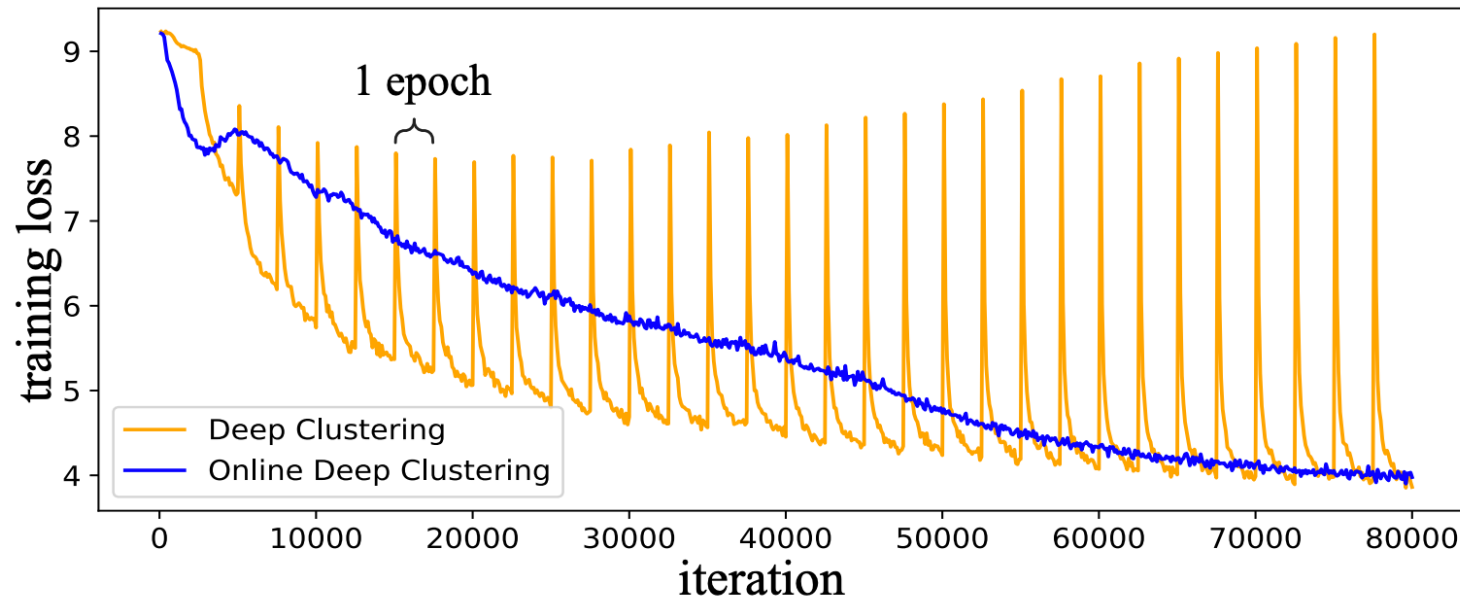
Deep clustering



Iteratively cluster deep features (offline) and use the cluster assignments as pseudo-labels to learn the parameters of the convnet

Clustering has the advantage of requiring little domain knowledge and no specific signal from the inputs

Problem with offline clustering

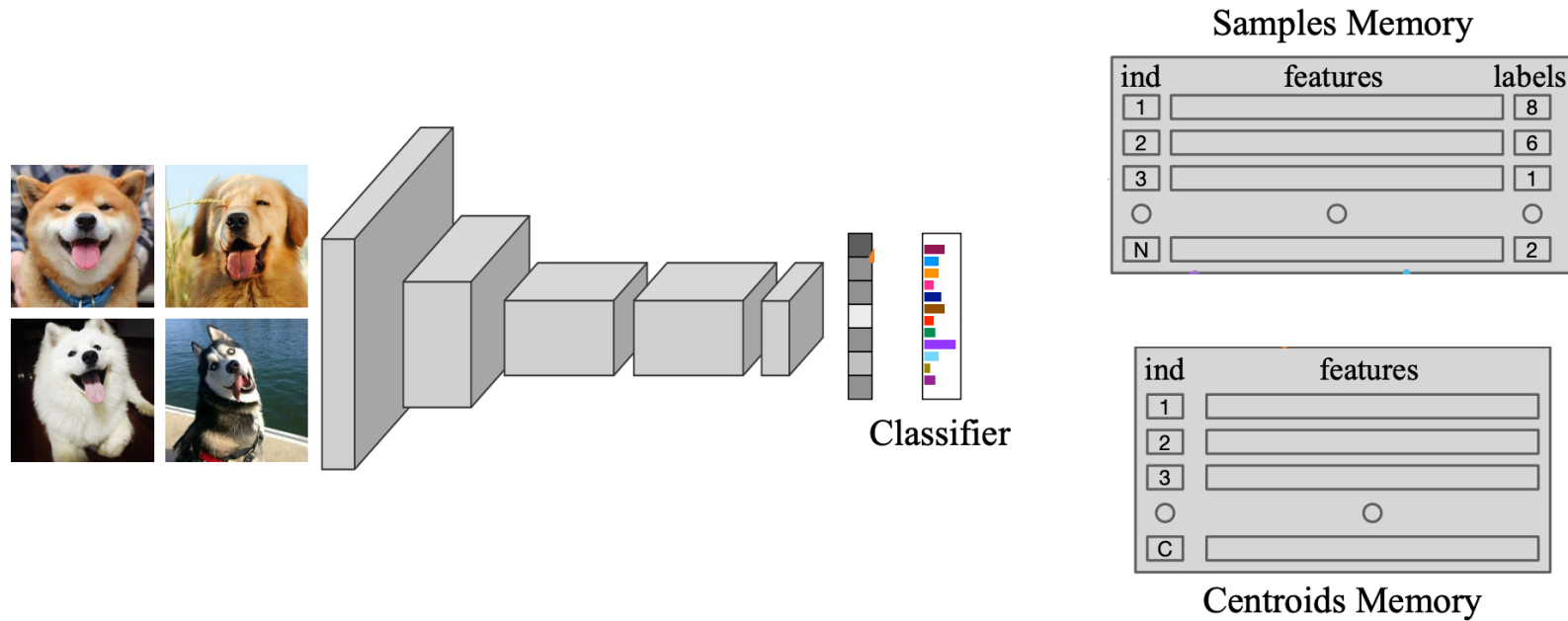


Offline clustering inevitably **permutes the assigned labels** in different epochs, i.e., even if some of the clusters do not change, their indices after clustering will be permuted randomly.

As a result, **parameters in the classifier cannot be inherited** from the last epoch and they have to be randomly initialized before each epoch.

The mechanism introduces training **instability** and exposes representations to a high risk of representation corruption

Online deep clustering

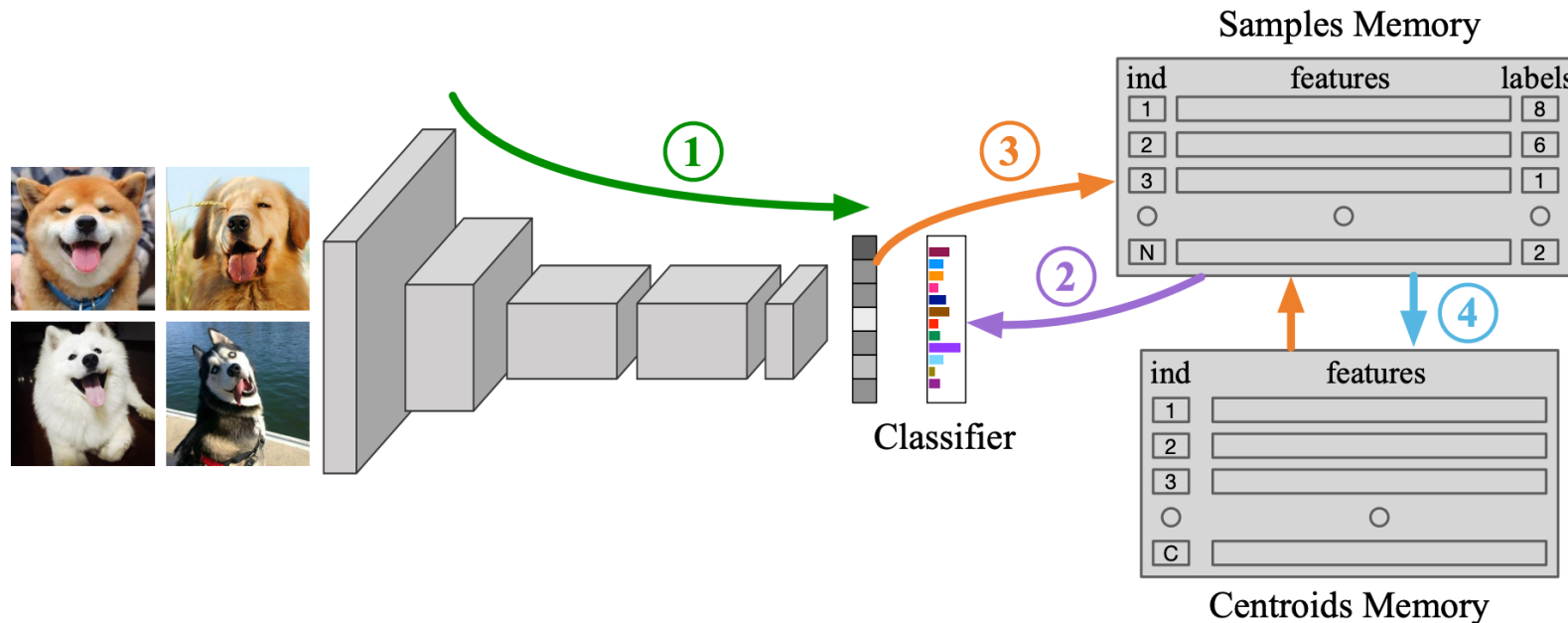


Samples memory stores features and pseudo-labels of the entire dataset

Centroids memory stores the features of class centroids, i.e., the mean feature of all samples in a class

A “class” here represents a temporary cluster that evolves continuously during training

Online deep clustering



1. Forward to obtain a compact feature vector;
2. Read labels from the samples memory and perform back-propagation to update the CNN;
3. Update samples memory by updating features and assigning new labels;
4. Update centroids memory by recomputing the involved centroids.

$$\left\{ \begin{array}{l} F_m(x) \leftarrow m \frac{f_\theta(x)}{\|f_\theta(x)\|_2} + (1 - m) F_m(x) \\ \min_{y \in \{1, \dots, C\}} \|F_m(x) - C_y\|_2^2 \end{array} \right.$$

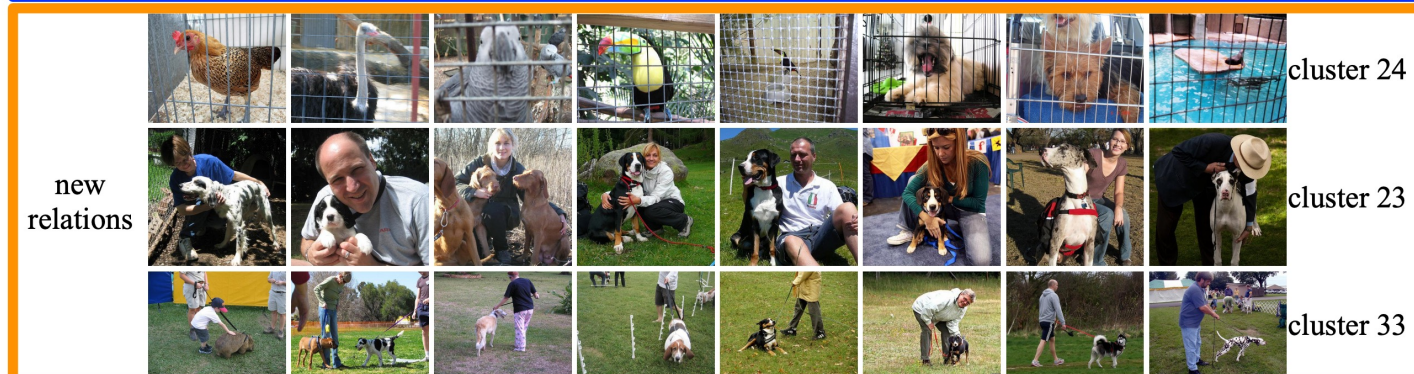
Clusters discovered



“hand” or “feet” is not a category in ImageNet annotations



“animal in cage”, “person holds dog” and “person leads dog with a rope”

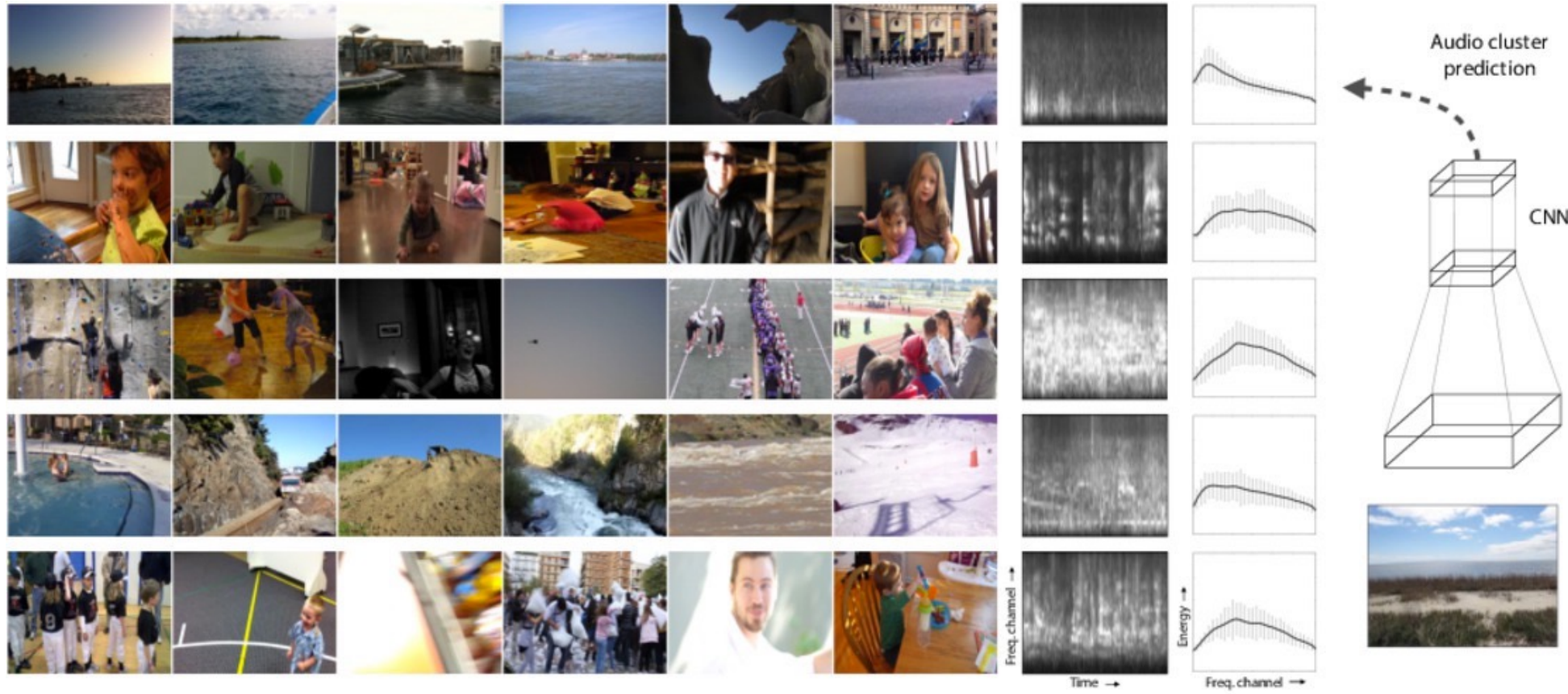


Cross-modality Methods

Ambient sound

- Acoustics -> RGB

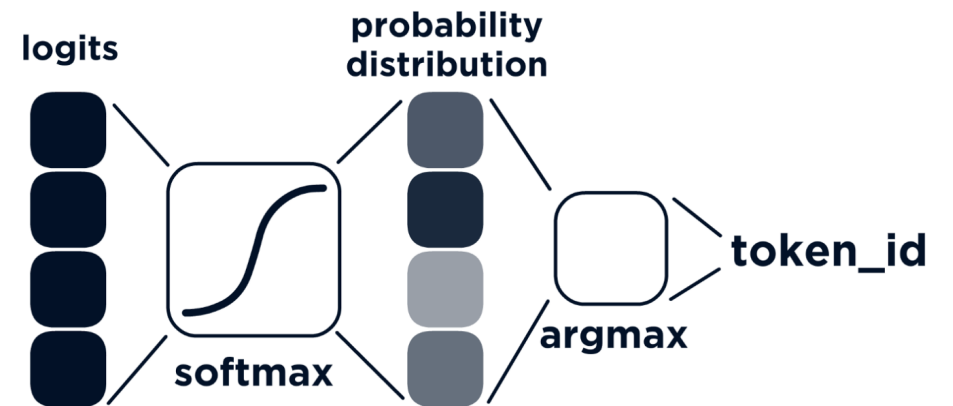
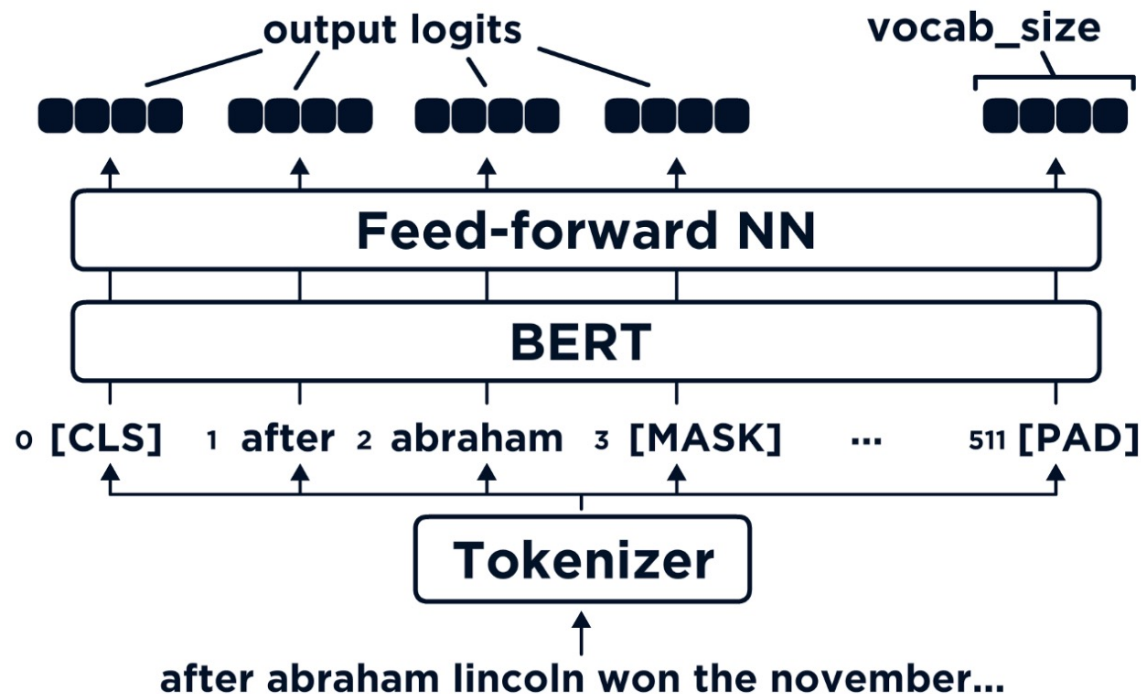
the response to a bank of bandpass modulation filters (sorted left-to-right in increasing order of frequency)



Masked Image Modeling

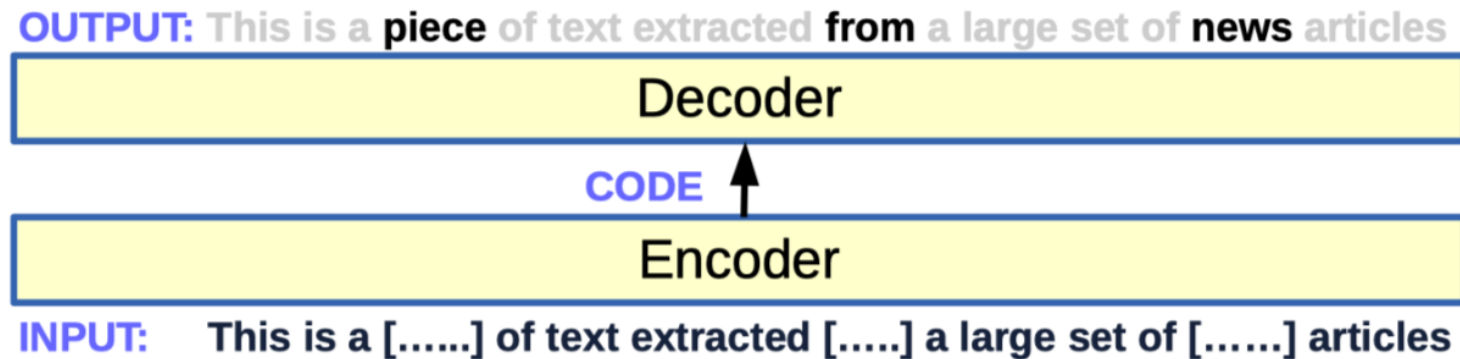
Masked Language Modeling

Mask out some tokens within a text and then predict the masked content given the visible text

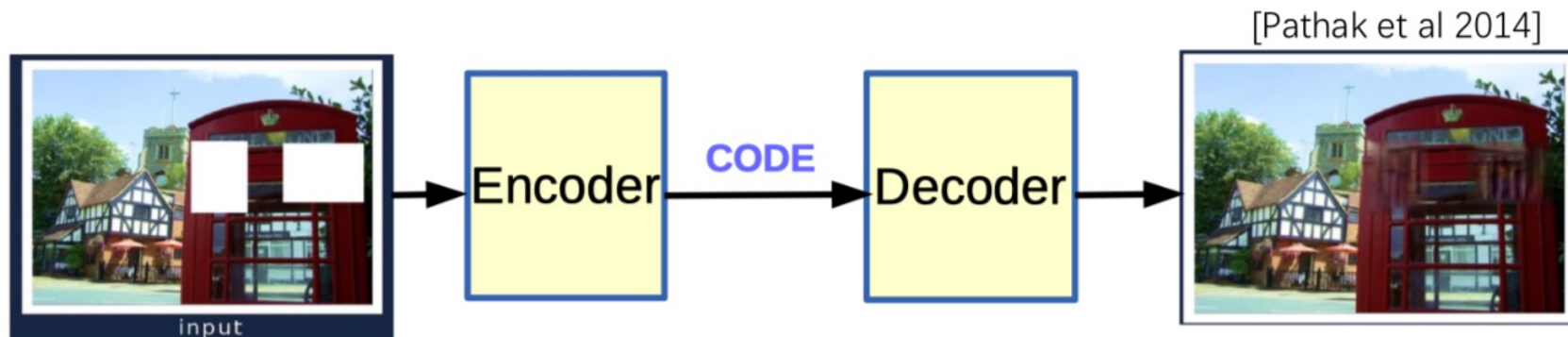


Masked Language Modeling

- Natural Language Processing: works great!

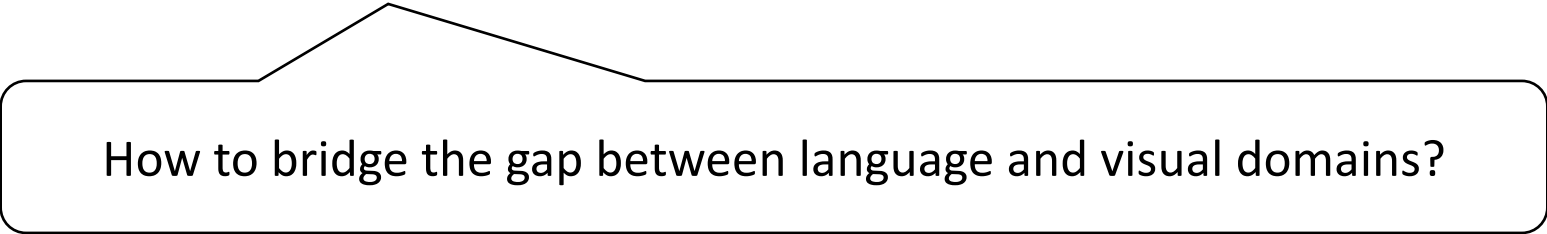


- Image Recognition: works so-so...



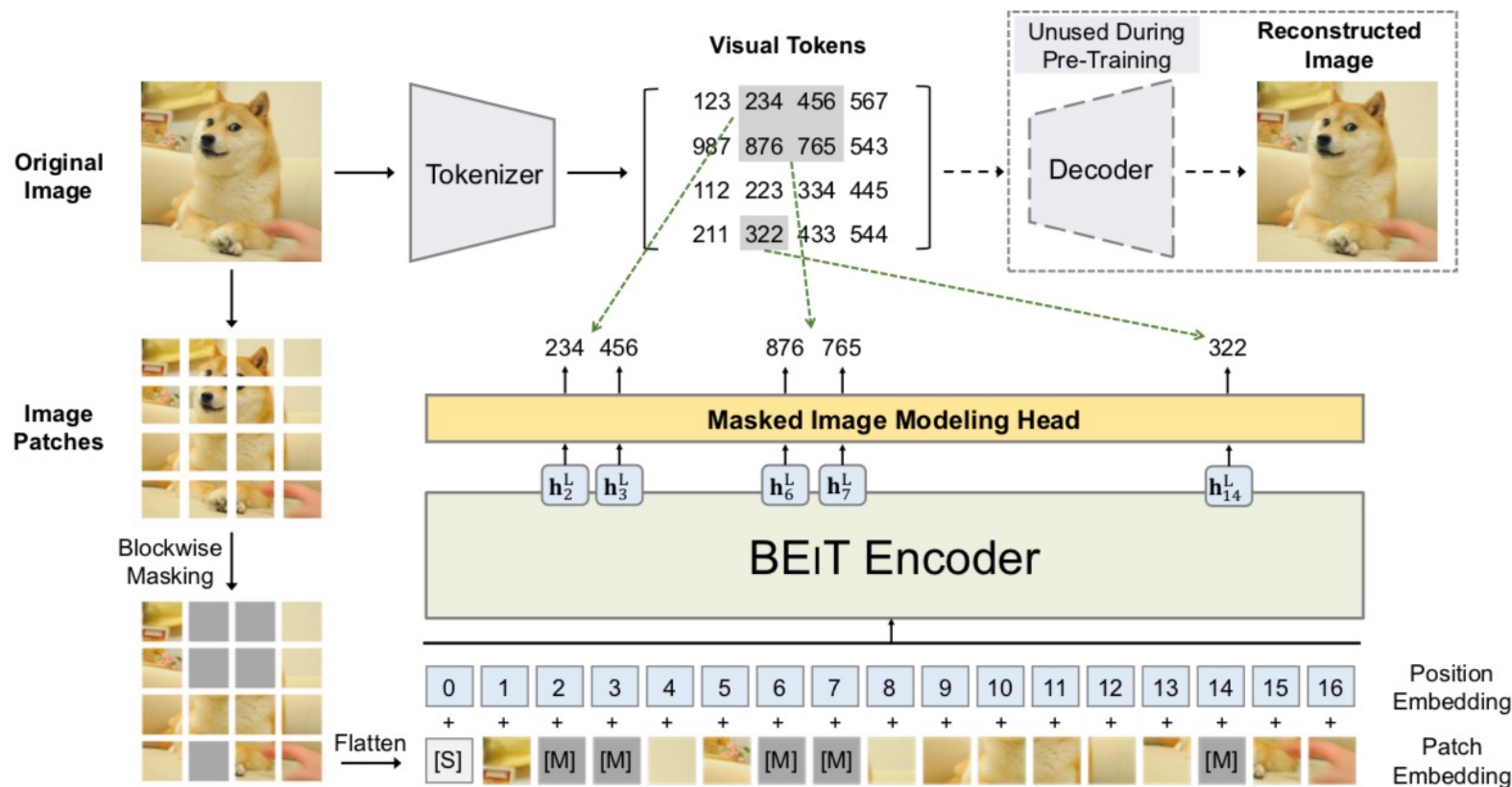
Masked Language Modeling

- Differences between language and visual modalities
 - Images exhibit stronger **locality**: pixels near to each other are highly correlated
 - Visual signals are **raw and low-level**, while text tokens contain **high-level** concepts
 - Visual signals are **continuous**, while text tokens are **discrete**



How to bridge the gap between language and visual domains?

BEiT



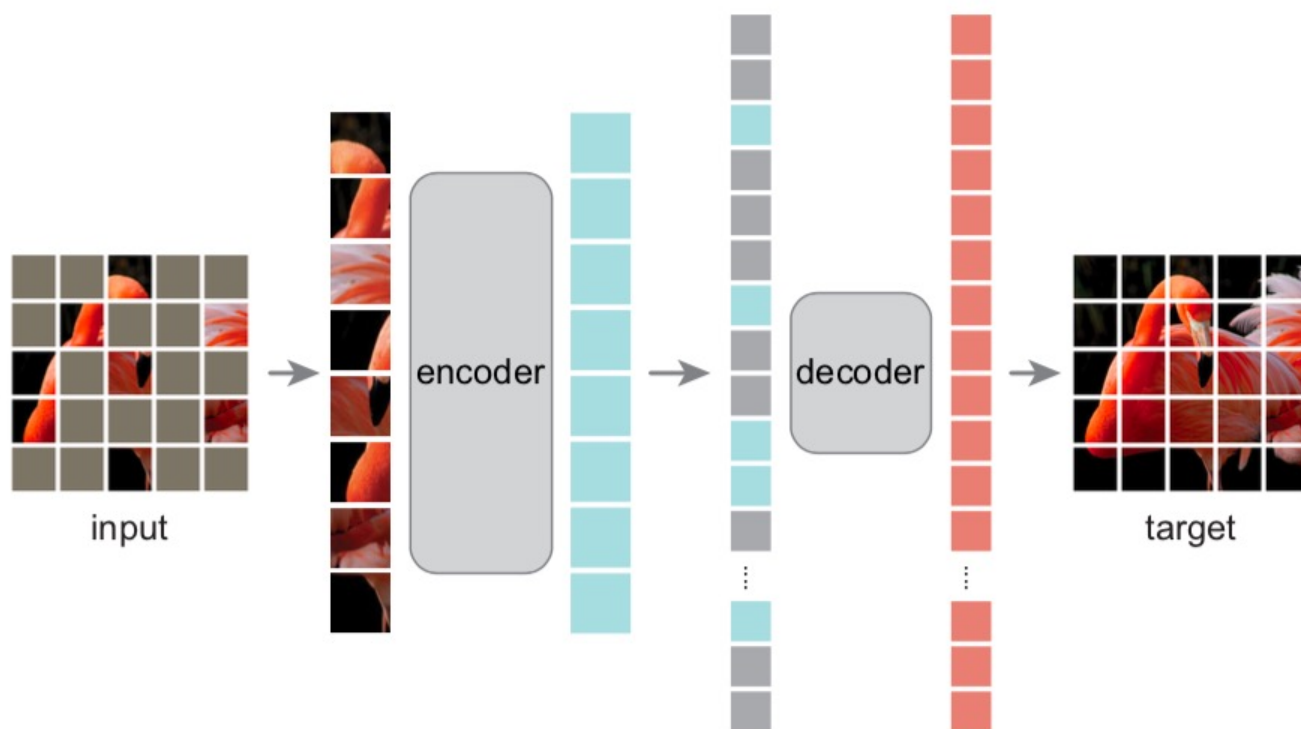
Before pre-training, we learn an “image tokenizer” via autoencoding-style reconstruction, where an image is tokenized into discrete visual tokens according to the learned vocabulary.

During pre-training, each image has two views, i.e., image patches, and visual tokens.

We randomly mask some proportion of image patches (gray patches in the figure) and replace them with a special mask embedding [M].

Then the patches are fed to a backbone vision Transformer. The pre-training task aims at predicting the visual tokens of the original image based on the encoding vectors of the corrupted image

MAE



During pre-training, a large random subset of image patches (*e.g.*, 75%) is masked out. The encoder is applied to the small subset of *visible patches*.

Mask tokens are introduced *after* the encoder, and the full set of encoded patches and mask tokens is processed by a small decoder that reconstructs the original image in pixels.

After pre-training, the decoder is discarded and the encoder is applied to uncorrupted images (full sets of patches) for recognition tasks.